

Входной конвейер в коде (scaffolded)

Self-attention: query, key, value и causal mask

Источники: С. Рашка. Создаём большую языковую модель с нуля; Vaswani et al., 2017. Attention Is All You Need

Александр Чурсин
Руководитель группы
разработки в ГК «Норильский
никель»



Проверка связи

Если у вас нет звука:

- убедитесь, что на вашем устройстве и на колонках включён звук
- обновите страницу вебинара (или закройте страницу и заново присоединитесь к вебинару)
- откройте вебинар в другом браузере
- перезагрузите компьютер (ноутбук) и заново попытайтесь зайти

Поставьте в чат:

-  если меня видно и слышно
-  если нет

Рекомендации

→ Если смотрите с компьютера

- Используйте браузеры Google Chrome или Microsoft Edge
- Если есть проблемы с изображением или звуком, обновите страницу — **F5**

→ Если смотрите с мобильного телефона или планшета

- Перейдите с мобильного интернет-соединения на **Wi-Fi**
- Если есть проблемы с изображением или звуком, перезапустите приложение на телефоне

Правила участия

- 1 Приготовьте блокнот и ручку, чтобы записывать важные мысли и идеи
- 2 Продолжительность вебинара — 1 час 20 минут
- 3 Вы можете писать свои вопросы в чате
- 4 Запись вебинара будет доступна в личном кабинете Нетологии
- 5 Оффтоп обсуждения можно продолжить в учебном чате в ВК

Александр Чурсин

О спикере:

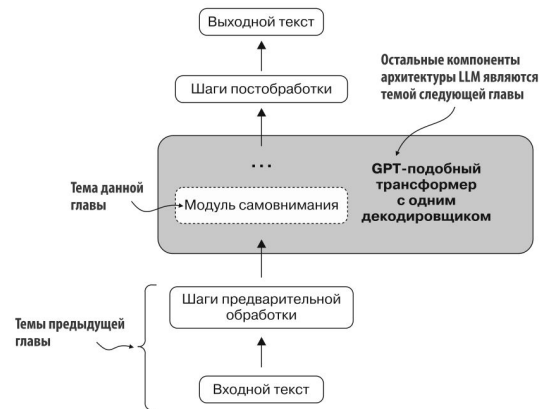
- TeamLead NLP в Норникеле
- Стажёр → тимлид команды из 10 за 5 лет
- Бакалавриат «Лингвистика»
- Магистр «Инженерия машинного обучения»
- Мультиагентные системы и дообучение
- Домены: мебель, туризм, медиа, металлургия



Где мы остановились

Занятие 2 собрало вход: матрица X из L строк по d координат

```
1 tok = token_embedding_layer(ids)
2 pos = pos_embedding_layer(torch.arange(L))
3 X = tok + pos
4 print(X.shape) # torch.Size([6, 3])
```



- $L = 6$ токенов, $d = 3$ координаты — эти два числа стоят во всех формах занятия

План занятия

- 1 **Что считает внимание**
- 2 Внимание без параметров
- 3 Три матрицы: query, key, value
- 4 Causal mask и dropout
- 5 Проверка и отладка
- 6 Позиция внутри внимания

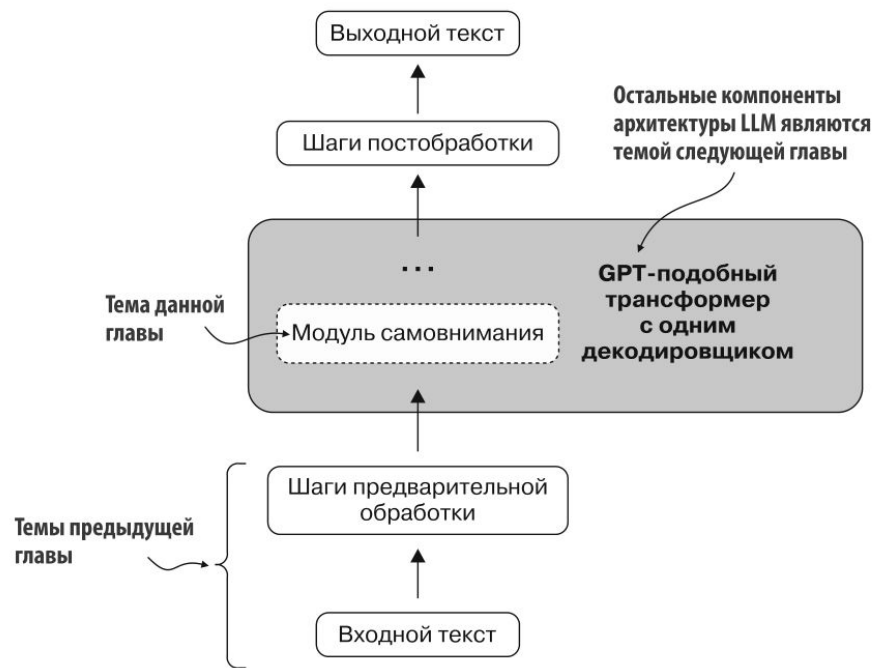
Цели занятия

- Понять, что считает внимание: три шага и context vector
- Ввести три матрицы: query, key и value
- Понять, зачем делить на корень из d_k
- Закрыть будущее causal mask и добавить dropout
- Научиться ловить типовые ошибки внимания
- Внести позицию в query и key: RoPE и ALiBi

Каждый токен смотрит на все токены фразы

Self-attention оценивает вклад каждой позиции той же последовательности при вычислении нового представления.

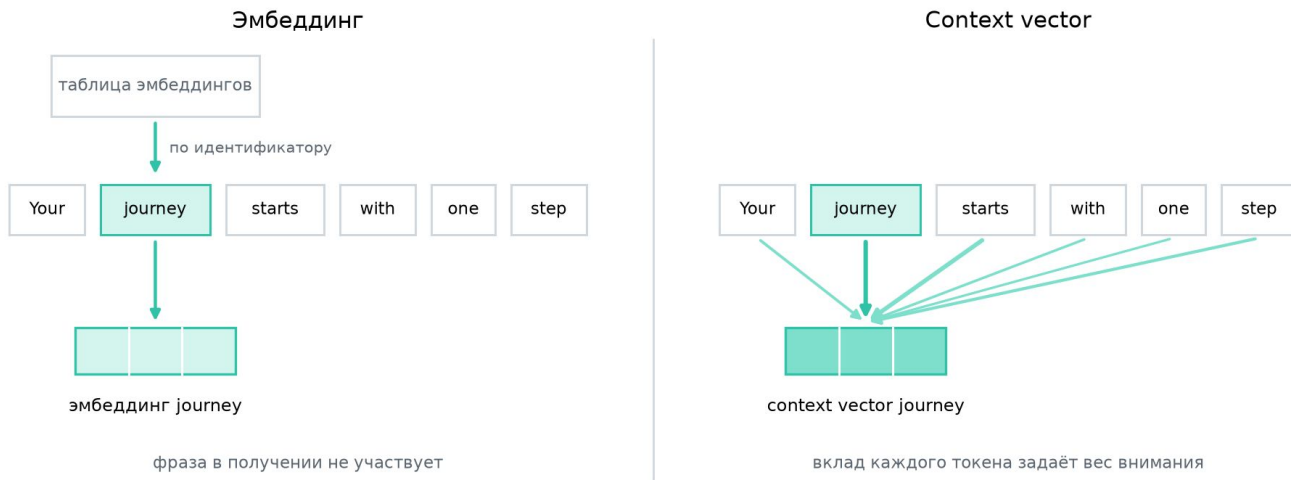
Внимание идёт после подготовки входа



- Ниже модуля — вход, собранный на занятии 2
- Выше — остальной блок и выходной слой
- Вход модуля — $X: L = 6$ токенов на $d = 3$ координаты

- Self-attention — каждый токен сравнивается со всеми токенами той же последовательности

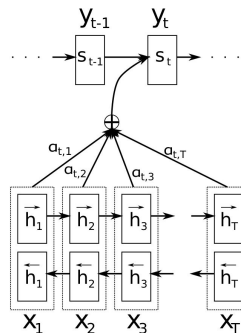
Два способа получить вектор токена



- **Эмбеддинг** — строка таблицы по идентификатору токена
- **Context vector** — смесь всех токенов фразы с весами внимания
- Self-attention — каждый токен сравнивается со всеми токенами той же последовательности
- context vector — выход внимания для токена: смесь value с весами внимания

Внимание старше трансформера

Слева 2014 год, справа 2017-й: идея весов та же, счёт другой

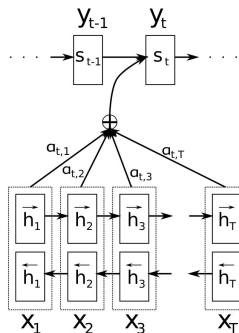


вход — немецкий, выход — английский

- Self-attention — каждый token сравнивается со всеми токенами той же последовательности
context vector — выход внимания для токена: смесь value с весами внимания

Внимание старше трансформера

Слева 2014 год, справа 2017-й: идея весов та же, счёт другой



вход — немецкий, выход — английский

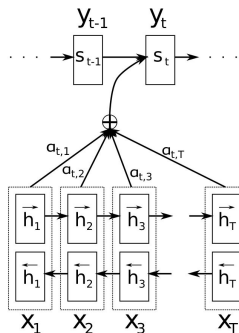
$$e_{ij} = v^T \tanh(W_s s_{i-1} + W_h h_j)$$

$e(i, j)$ — важность токена j для шага i ; s и h — состояния декодировщика и кодировщика

- Self-attention — каждый токен сравнивается со всеми токенами той же последовательности
context vector — выход внимания для токена: смесь value с весами внимания

Внимание старше трансформера

Слева 2014 год, справа 2017-й: идея весов та же, счёт другой



вход — немецкий, выход — английский

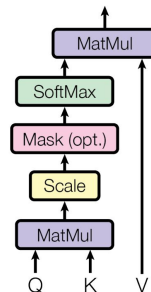
$$e_{ij} = v^T \tanh(W_s s_{i-1} + W_h h_j)$$

$e(i, j)$ — важность токена j для шага i ; s и h — состояния декодировщика и кодировщика

- Self-attention — каждый токен сравнивается со всеми токенами той же последовательности
context vector — выход внимания для токена: смесь value с весами внимания

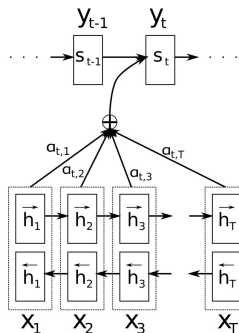
Источник: Bahdanau et al., 2014, рис. 1, arXiv:1409.0473; Vaswani et al., 2017, рис. 2, arXiv:1706.03762

Scaled Dot-Product Attention



Внимание старше трансформера

Слева 2014 год, справа 2017-й: идея весов та же, счёт другой

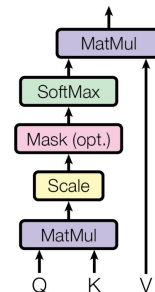


вход — немецкий, выход — английский

$$e_{ij} = v^T \tanh(W_s s_{i-1} + W_h h_j)$$

$e(i, j)$ — важность токена j для шага i ; s и h — состояния декодировщика и кодировщика

Scaled Dot-Product Attention



$$e_{ij} = \langle q_i, k_j \rangle$$

$e(i, j)$ — та же важность; q и k — векторы query и key; i и j — номера токенов

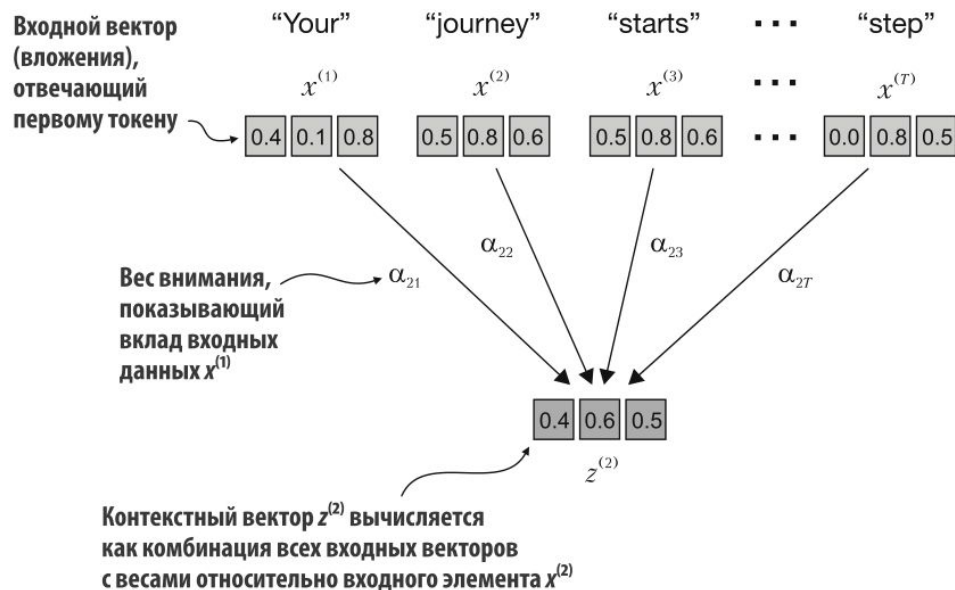
- Self-attention — каждый токен сравнивается со всеми токенами той же последовательности
context vector — выход внимания для токена: смесь value с весами внимания

План занятия

- 1 Что считает внимание
- 2 Внимание без параметров**
- 3 Три матрицы: query, key, value
- 4 Causal mask и dropout
- 5 Проверка и отладка
- 6 Позиция внутри внимания

Три шага на шести токенах

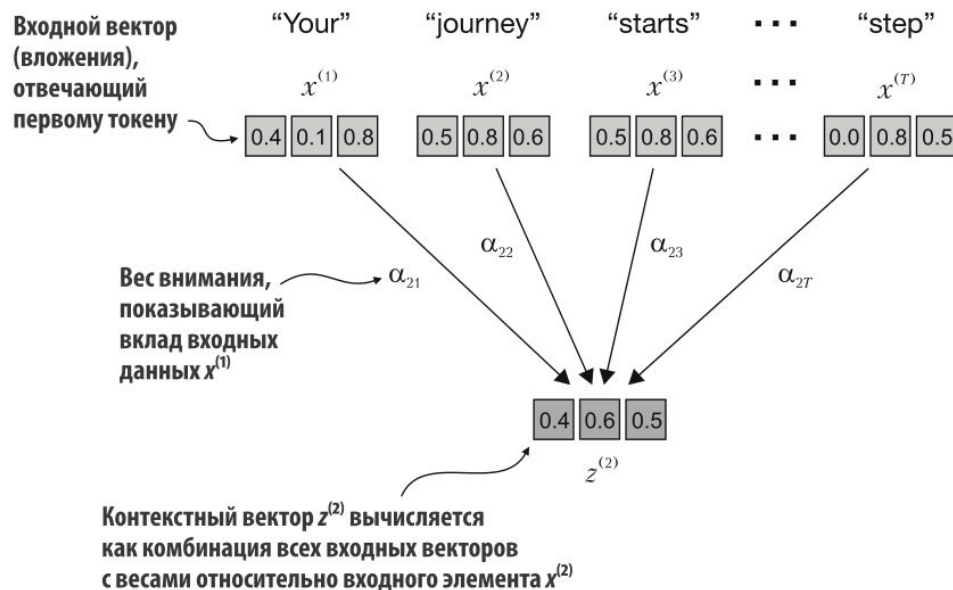
Your journey starts with one step: $X = 6 \text{ строк} \times 3 \text{ числа}$



- Self-attention — каждый токен сравнивается со всеми токенами той же последовательности
- context vector — выход внимания для токена: смесь value с весами внимания

Три шага на шести токенах

Your journey starts with one step: $X = 6 \text{ строк} \times 3 \text{ числа}$

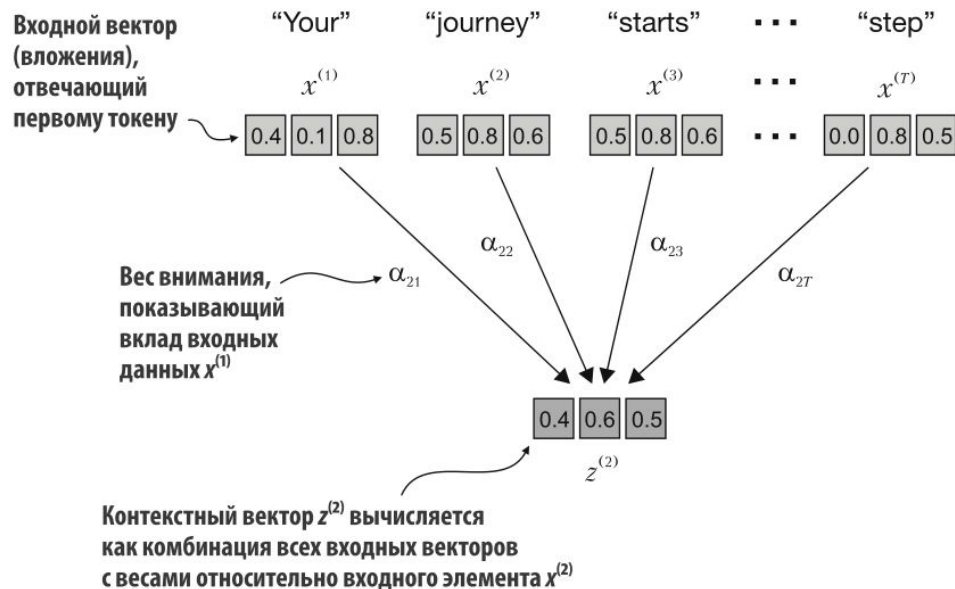


- Вход: шесть токенов по три числа

- Self-attention — каждый токен сравнивается со всеми токенами той же последовательности
context vector — выход внимания для токена: смесь value с весами внимания

Три шага на шести токенах

Your journey starts with one step: $X = 6 \text{ строк} \times 3 \text{ числа}$



- Вход: шесть токенов по три числа
- Выход: **$z(2)$** — смесь всех шести

- Self-attention — каждый токен сравнивается со всеми токенами той же последовательности
- context vector — выход внимания для токена: смесь value с весами внимания

Вход: шесть строк по три числа

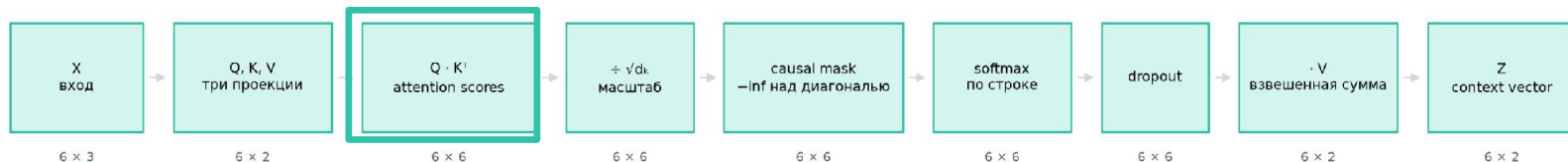
Шаг 1

Числа учебные: взяты как есть, смысла в них искать не нужно

```
1 import torch
2
3 inputs = torch.tensor(
4     [[0.43, 0.15, 0.89], # Your    (x^1)
5      [0.55, 0.87, 0.66], # journey (x^2)
6      [0.57, 0.85, 0.64], # starts  (x^3)
7      [0.22, 0.58, 0.33], # with   (x^4)
8      [0.77, 0.25, 0.10], # one    (x^5)
9      [0.05, 0.80, 0.55]] # step    (x^6)
10 )
```

Шаг первый: attention scores

Шаг 1



$$\omega_{ij} = \langle x_i, x_j \rangle$$

ω (омега) — attention score пары до softmax; $x(i)$ и $x(j)$ — эмбединги токенов с номерами i и j

- Скалярное произведение растёт, когда векторы длиннее и сонаправлены
- Пока query и key — сам эмбединг x

Внимание 2014 года работает и на картинках и звуке: механизм не привязан к тексту, меняется только вход.

- Self-attention — каждый токен сравнивается со всеми токенами той же последовательности
- context vector — выход внимания для токена: смесь value с весами внимания

Источник: скрипт `figures/src/1_3_21.py`

Шесть attention scores для journey

Шаг 1

Цикл по токенам и та же операция руками

```
1 query = inputs[1]          # второй токен, journey, служит запросом
2 attn_scores_2 = torch.empty(inputs.shape[0])
3 for i, x_i in enumerate(inputs):
4     attn_scores_2[i] = torch.dot(x_i, query)
5 print(attn_scores_2)
6 # tensor([0.9544, 1.4950, 1.4754, 0.8434, 0.7070, 1.0865])
7
8 res = 0.
9 for idx, element in enumerate(inputs[0]):
10     res += inputs[0][idx] * query[idx]
11 print(res)                  # tensor(0.9544)
12 print(torch.dot(inputs[0], query)) # tensor(0.9544)
```

Делим на сумму: доли, а не числа

Шаг 2

Простейшая нормировка attention scores

```
1 atn_weights_2_tmp = atn_scores_2 / atn_scores_2.sum()
2 print("Attention weights:", atn_weights_2_tmp)
3 # tensor([0.1455, 0.2278, 0.2249, 0.1285, 0.1077, 0.1656])
4 print("Sum:", atn_weights_2_tmp.sum())    # tensor(1.0000)
```

Своя экспонента и torch.softmax

Одна функция вместо двух строк — и без переполнения

```
1 def softmax_naive(x):
2     return torch.exp(x) / torch.exp(x).sum(dim=0)
3
4 attn_weights_2_naive = softmax_naive(attn_scores_2)
5 print("Attention weights:", attn_weights_2_naive)
6 # Your journey starts with one step
7 # tensor([0.1385, 0.2379, 0.2333, 0.1240, 0.1082, 0.1581])
8 print("Sum:", attn_weights_2_naive.sum()) # tensor(1.)
9 attn_weights_2 = torch.softmax(attn_scores_2, dim=0)
10 print("Attention weights:", attn_weights_2)
11 # Your journey starts with one step
12 # tensor([0.1385, 0.2379, 0.2333, 0.1240, 0.1082, 0.1581])
13 print("Sum:", attn_weights_2.sum()) # tensor(1.)
```

Своя экспонента и torch.softmax

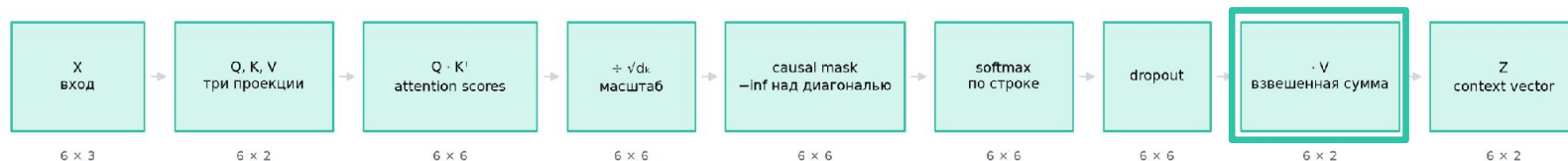
Шаг 2

Одна функция вместо двух строк — и без переполнения

```
1 def softmax_naive(x):
2     return torch.exp(x) / torch.exp(x).sum(dim=0)
3
4 attn_weights_2_naive = softmax_naive(attn_scores_2)
5 print("Attention weights:", attn_weights_2_naive)
6 # Your journey starts with one step
7 # tensor([0.1385, 0.2379, 0.2333, 0.1240, 0.1082, 0.1581])
8 print("Sum:", attn_weights_2_naive.sum()) # tensor(1.)
9 attn_weights_2 = torch.softmax(attn_scores_2, dim=0)
10 print("Attention weights:", attn_weights_2)
11 # Your journey starts with one step
12 # tensor([0.1385, 0.2379, 0.2333, 0.1240, 0.1082, 0.1581])
13 print("Sum:", attn_weights_2.sum()) # tensor(1.)
```

Шаг третий: context vector

Шаг 3



$$z_i = \sum_j \alpha_{ij} x_j$$

$z(i)$ — context vector токена i ; $\alpha(i, j)$ — вес внимания; $x(j)$ — эмбединг токена j

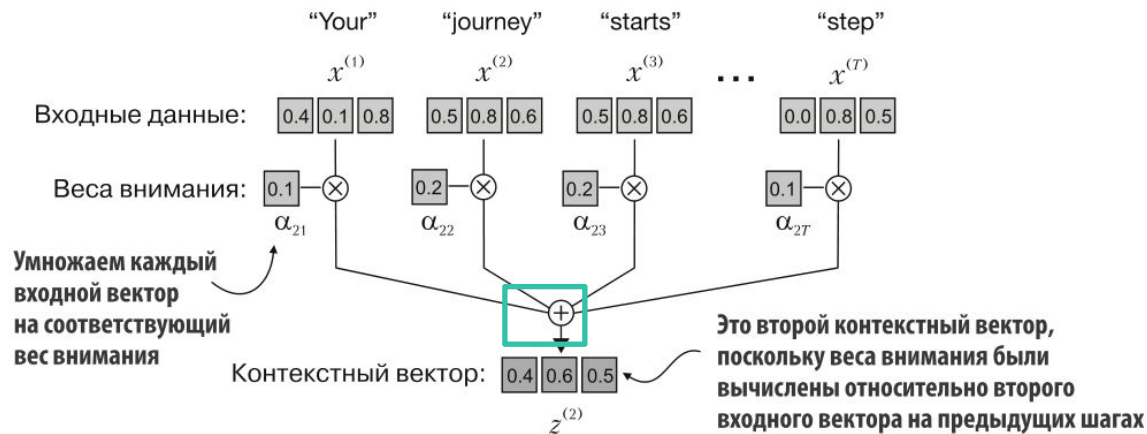
- Пока value — сам эмбединг x_j : смесь собирается из входных векторов

- Self-attention — каждый токен сравнивается со всеми токенами той же последовательности
context vector — выход внимания для токена: смесь value с весами внимания
веса внимания — attention scores после softmax; сумма по строке равна единице

Источник: скрипт `figures/src/1_3_21.py`

Context vector — взвешенная сумма

Шаг 3



- Знак $\otimes$ — умножение входного вектора на его вес внимания
- Знак $\oplus$ — сложение шести полученных векторов в **$z(2)$**

● Self-attention — каждый токен сравнивается со всеми токенами той же последовательности
context vector — выход внимания для токена: смесь value с весами внимания
веса внимания — attention scores после softmax; сумма по строке равна единице

Один цикл — и context vector готов

Шаг 3

Веса внимания journey умножаются на входы и складываются

```
1 query = inputs[1]                # второй токен по-прежнему запрос
2 context_vec_2 = torch.zeros(query.shape)
3 for i, x_i in enumerate(inputs):
4     context_vec_2 += attn_weights_2[i] * x_i
5 print(context_vec_2)
6 # tensor([0.4419, 0.6515, 0.5683])
```

Веса всех токенов — матрица $L \times L$

	Your	journey	starts	with	one	step
Your	0.20	0.20	0.19	0.12	0.12	0.14
journey	0.13	0.23	0.23	0.12	0.10	0.15
starts	0.13	0.23	0.23	0.12	0.11	0.15
with	0.14	0.20	0.20	0.14	0.12	0.17
one	0.15	0.19	0.19	0.13	0.18	0.12
step	0.13	0.21	0.21	0.14	0.09	0.18

Эта строка содержит веса внимания
(нормализованные показатели внимания),
вычисленные ранее

- Строка — токен, который смотрит
- Столбец — токен, на который смотрят
- Сумма по строке — **единица**

- Self-attention — каждый токен сравнивается со всеми токенами той же последовательности
- context vector — выход внимания для токена: смесь value с весами внимания
- веса внимания — attention scores после softmax; сумма по строке равна единице

Два цикла или одно умножение

Шаг 1

Элемент (i, j) — произведение i -й строки на j -ю

```
1  attn_scores = torch.empty(6, 6)
2  for i, x_i in enumerate(inputs):
3      for j, x_j in enumerate(inputs):
4          attn_scores[i, j] = torch.dot(x_i, x_j)
5  print(attn_scores)
6  # tensor([[[0.9995, 0.9544, 0.9422, 0.4753, 0.4576, 0.6310],
7  #         [0.9544, 1.4950, 1.4754, 0.8434, 0.7070, 1.0865],
8  #         [0.9422, 1.4754, 1.4570, 0.8296, 0.7154, 1.0605],
9  #         [0.4753, 0.8434, 0.8296, 0.4937, 0.3474, 0.6565],
10 #         [0.4576, 0.7070, 0.7154, 0.3474, 0.6654, 0.2935],
11 #         [0.6310, 1.0865, 1.0605, 0.6565, 0.2935, 0.9450]])
12
13 attn_scores = inputs @ inputs.T
14 print(attn_scores)      # та же матрица 6 × 6, что и с двумя циклами
```

Softmax по строкам: dim = -1

Шаг 2

Матрица весов внимания 6×6 для всех токенов

```
1 tokens = ["Your", "journey", "starts", "with", "one", "step"]
2 attn_weights = torch.softmax(attn_scores, dim=-1)
3 print(f"{':8}', " ".join(f'{t:>7}' for t in tokens)) # столбцы — на кого смотрят
4 for tok, row in zip(tokens, attn_weights):
5     print(f'{tok:8}', " ".join(f'{w:7.4f}' for w in row)) # строка — кто смотрит
6 #       Your journey starts with one step
7 # Your   0.2098 0.2006 0.1981 0.1242 0.1220 0.1452
8 # journey 0.1385 0.2379 0.2333 0.1240 0.1082 0.1581
9 # starts  0.1390 0.2369 0.2326 0.1242 0.1108 0.1565 ← на journey 0,2369, на себя 0,2326
10 # with   0.1435 0.2074 0.2046 0.1462 0.1263 0.1720
11 # one    0.1526 0.1958 0.1975 0.1367 0.1879 0.1295
12 # step   0.1385 0.2184 0.2128 0.1420 0.0988 0.1896
```

$\langle \text{starts}, \text{starts} \rangle = 1,457$ · $\langle \text{starts}, \text{journey} \rangle = 1,4754$

Softmax по строкам: dim = -1

Шаг 2

Матрица весов внимания 6×6 для всех токенов

```
1 tokens = ["Your", "journey", "starts", "with", "one", "step"]
2 attn_weights = torch.softmax(attn_scores, dim=-1)
3 print(f"{':8}', " ".join(f'{t:>7}' for t in tokens)) # столбцы — на кого смотрят
4 for tok, row in zip(tokens, attn_weights):
5     print(f'{tok:8}', " ".join(f'{w:7.4f}' for w in row)) # строка — кто смотрит
6 #       Your journey starts with one step
7 # Your   0.2098 0.2006 0.1981 0.1242 0.1220 0.1452
8 # journey 0.1385 0.2379 0.2333 0.1240 0.1082 0.1581
9 # starts  0.1390 0.2369 0.2326 0.1242 0.1108 0.1565 ← на journey 0,2369, на себя 0,2326
10 # with   0.1435 0.2074 0.2046 0.1462 0.1263 0.1720
11 # one    0.1526 0.1958 0.1975 0.1367 0.1879 0.1295
12 # step   0.1385 0.2184 0.2128 0.1420 0.0988 0.1896
```

$\langle \text{starts}, \text{starts} \rangle = 1,457$ · $\langle \text{starts}, \text{journey} \rangle = 1,4754$

Суммы строк ловят перепутанную ось

Шаг 2

Форма та же, исключения нет, смысл другой

```
1 row_2_sum = sum([0.1385, 0.2379, 0.2333, 0.1240, 0.1082, 0.1581])
2 print("Row 2 sum:", row_2_sum)          # 1.0
3 print("All row sums:", attn_weights.sum(dim=-1))
4 # tensor([1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000])
5 assert torch.allclose(attn_weights.sum(dim=-1),
6                       torch.ones(attn_weights.shape[0])) # проверка оси
7
8 wrong = torch.softmax(attn_scores, dim=0) # ось перепутана
9 print(wrong.shape)                       # torch.Size([6, 6]) — форма та же
10 print("row sums:", wrong.sum(dim=-1))
11 # tensor([0.9220, 1.2970, 1.2788, 0.7974, 0.7540, 0.9508])
12 print("col sums:", wrong.sum(dim=0))
13 # tensor([1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000])
```

Первая из трёх типовых поломок внимания: ось softmax перепутана.

Все context vector одним умножением

Шаг 3

Веса внимания умножаем на входы

```
1 all_context_vecs = attn_weights @ inputs
2 print(all_context_vecs)
3 # tensor([[[0.4421, 0.5931, 0.5790],
4 #          [0.4419, 0.6515, 0.5683],
5 #          [0.4431, 0.6496, 0.5671],
6 #          [0.4304, 0.6298, 0.5510],
7 #          [0.4671, 0.5910, 0.5266],
8 #          [0.4177, 0.6503, 0.5645]]])
9
10 print("Previous 2nd context vector:", context_vec_2)
11 # tensor([0.4419, 0.6515, 0.5683]) — совпадает со второй строкой
```

Соседи сменились — вектор другой

Шаг 3

Эмбеddинг journey не тронут, окружение изменено

```
1 alt = inputs.clone()
2 alt[0] = inputs[5] # вместо Your ставим вектор step: соседи journey изменились
3 alt_context = torch.softmax(alt @ alt.T, dim=-1) @ alt
4
5 print(inputs[1], alt[1]) # journey тот же: tensor([0.5500, 0.8700, 0.6600])
6 print(all_context_vecs[1]) # было: tensor([0.4419, 0.6515, 0.5683])
7 print(alt_context[1]) # стало: tensor([0.3827, 0.7427, 0.5218])
```

Координаты здесь порядка 0,5 — сдвиг составил больше десятой доли значения

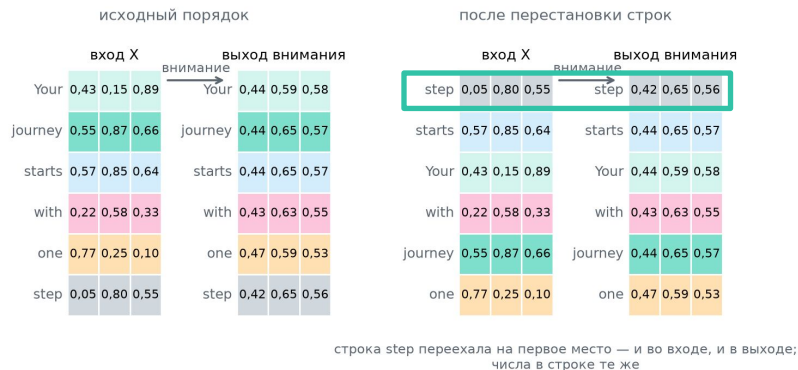
Переставим строки входа

Те же три операции на переставленной матрице

```
1 perm = torch.tensor([5, 2, 0, 3, 1, 4])      # новый порядок токенов
2 inputs_perm = inputs[perm]    # step, starts, Your, with, journey, one
3
4 scores_perm = inputs_perm @ inputs_perm.T
5 weights_perm = torch.softmax(scores_perm, dim=-1)
6 context_perm = weights_perm @ inputs_perm
7
8 print(torch.allclose(context_perm, all_context_vecs[perm])) # True
```

Порядок строк механизм не различает

Эквивариантность: переставили вход — переставился выход

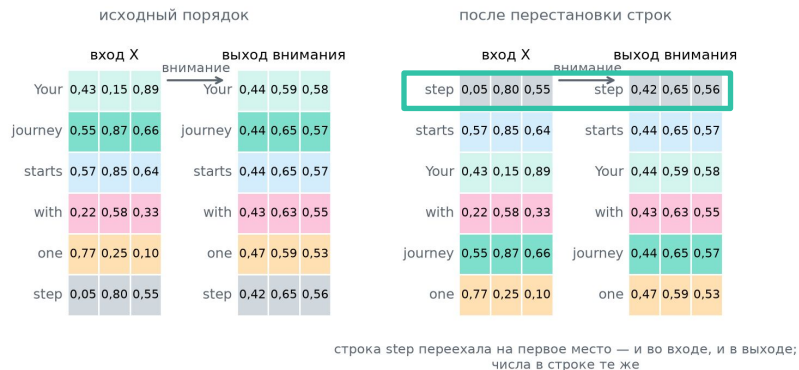


- Self-attention — каждый токен сравнивается со всеми токенами той же последовательности
- context vector — выход внимания для токена: смесь value с весами внимания
- веса внимания — attention scores после softmax; сумма по строке равна единице

Источник: Рис. 1.3.6 · своя схема, скрипт figures/src/1_3_6.py; числа книжного примера

Порядок строк механизм не различает

Эквивариантность: переставили вход — переставился выход



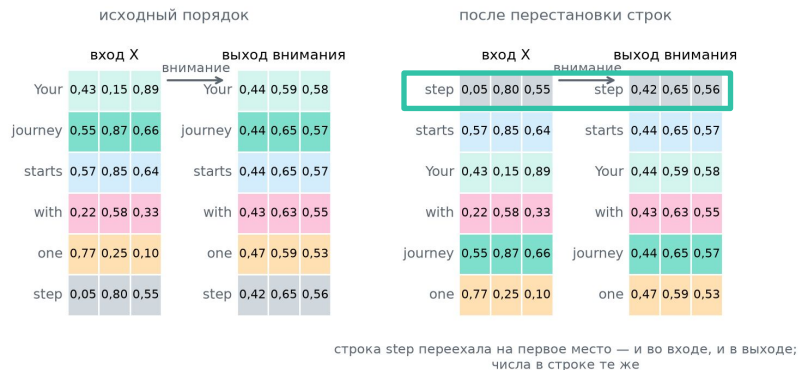
- Строка step переехала наверх и во входе, и в выходе — числа те же

- Self-attention — каждый токен сравнивается со всеми токенами той же последовательности
context vector — выход внимания для токена: смесь value с весами внимания
веса внимания — attention scores после softmax; сумма по строке равна единице

Источник: Рис. 1.3.6 · своя схема, скрипт figures/src/1_3_6.py; числа книжного примера

Порядок строк механизм не различает

Эквивариантность: переставили вход — переставился выход



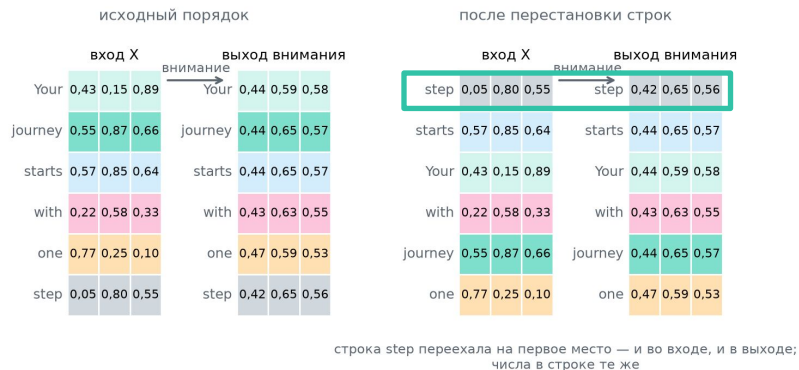
- Строка step переехала наверх и во входе, и в выходе — числа те же
- Как таблица без порядка строк: читается содержимое, а не номер

- Self-attention — каждый токен сравнивается со всеми токенами той же последовательности
context vector — выход внимания для токена: смесь value с весами внимания
веса внимания — attention scores после softmax; сумма по строке равна единице

Источник: Рис. 1.3.6 · своя схема, скрипт figures/src/1_3_6.py; числа книжного примера

Порядок строк механизм не различает

Эквивариантность: переставили вход — переставился выход



- Строка step переехала наверх и во входе, и в выходе — числа те же
- Как таблица без порядка строк: читается содержимое, а не номер
- Это **эквивариантность**, а не инвариантность: выход не остался прежним

- Self-attention — каждый токен сравнивается со всеми токенами той же последовательности
context vector — выход внимания для токена: смесь value с весами внимания
веса внимания — attention scores после softmax; сумма по строке равна единице

Источник: Рис. 1.3.6 · своя схема, скрипт figures/src/1_3_6.py; числа книжного примера

Ни одна операция не смотрит на номер

$$(PX)(PX)^{\top} = P (XX^{\top}) P^{\top}$$

X — матрица входа: строка на токен; P (пэ) — матрица перестановки, PX меняет порядок строк; $\top$ — транспонирование

- P (пэ) — матрица перестановки: PX меняет порядок строк X
- Матрица весов переезжает так же: $A \rightarrow P A P^{\top}$
- Обратная перестановка возвращает исходный порядок

● Self-attention — каждый токен сравнивается со всеми токенами той же последовательности
context vector — выход внимания для токена: смесь value с весами внимания
веса внимания — attention scores после softmax; сумма по строке равна единице

Ни одна операция не смотрит на номер

$$(PX)(PX)^{\top} = P (XX^{\top}) P^{\top}$$

X — матрица входа: строка на токен; P (пэ) — матрица перестановки, PX меняет порядок строк; $\top$ — транспонирование

$$PAP^{\top}PX = PAX, \quad \boxed{P^{\top}P = I}$$

I — единичная матрица: на диагонали единицы, вне её нули, умножение на неё ничего не меняет

- P (пэ) — матрица перестановки: PX меняет порядок строк X
- Матрица весов переезжает так же: $A \rightarrow P A P^{\top}$
- Обратная перестановка возвращает исходный порядок

● Self-attention — каждый токен сравнивается со всеми токенами той же последовательности
context vector — выход внимания для токена: смесь value с весами внимания
веса внимания — attention scores после softmax; сумма по строке равна единице

Ни одна операция не смотрит на номер

$$(PX)(PX)^{\top} = P (XX^{\top}) P^{\top}$$

X — матрица входа: строка на токен; P (пэ) — матрица перестановки, PX меняет порядок строк; $\top$ — транспонирование

$$PAP^{\top}PX = PAX, \quad \boxed{P^{\top}P = I}$$

I — единичная матрица: на диагонали единицы, вне её нули, умножение на неё ничего не меняет

- P (пэ) — матрица перестановки: PX меняет порядок строк X
- Матрица весов переезжает так же: $A \rightarrow P A P^{\top}$
- Обратная перестановка возвращает исходный порядок

- Self-attention — каждый токен сравнивается со всеми токенами той же последовательности
context vector — выход внимания для токена: смесь value с весами внимания
веса внимания — attention scores после softmax; сумма по строке равна единице

План занятия

- 1 Что считает внимание
- 2 Внимание без параметров
- 3 Три матрицы: query, key, value**
- 4 Causal mask и dropout
- 5 Проверка и отладка
- 6 Позиция внутри внимания

Обучение матричного внимания

Полное название механизма — scaled dot-product attention

Упрощённое внимание сравнивало сами эмбединги: обучать в нём нечего.

$$q_i = W_q x_i, \quad k_j = W_k x_j, \quad v_j = W_v x_j$$

q, k, v — векторы query, key, value; W с индексом — обучаемая матрица; $x(i)$ — эмбединг токена

- Self-attention — каждый токен сравнивается со всеми токенами той же последовательности
- context vector — выход внимания для токена: смесь value с весами внимания
- веса внимания — attention scores после softmax; сумма по строке равна единице

Без матриц внимание ничему не учится

Полное название механизма — scaled dot-product attention

Упрощённое внимание сравнивало сами эмбединги: обучать в нём нечего.

$$q_i = W_q x_i, \quad k_j = W_k x_j, \quad v_j = W_v x_j$$

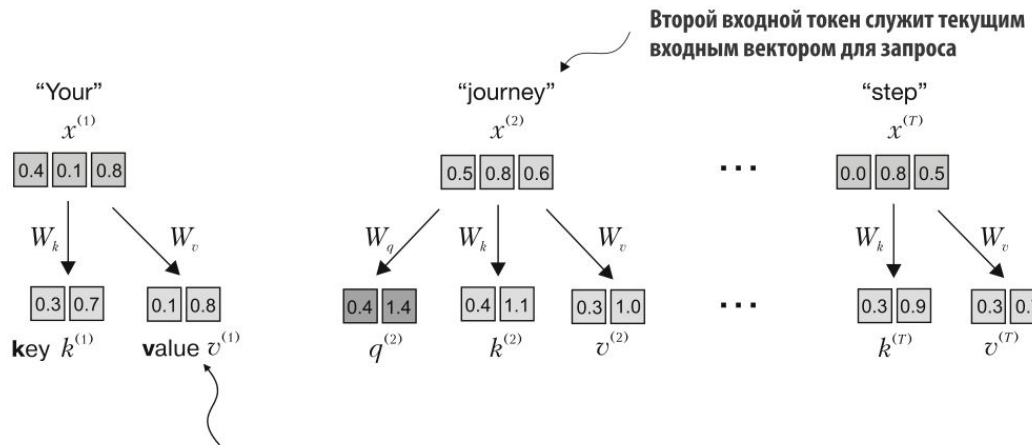
q, k, v — векторы query, key, value; W с индексом — обучаемая матрица; $x(i)$ — эмбединг токена

- Обучаются только эти три матрицы — сам механизм параметров не имеет

- Self-attention — каждый токен сравнивается со всеми токенами той же последовательности
context vector — выход внимания для токена: смесь value с весами внимания
веса внимания — attention scores после softmax; сумма по строке равна единице

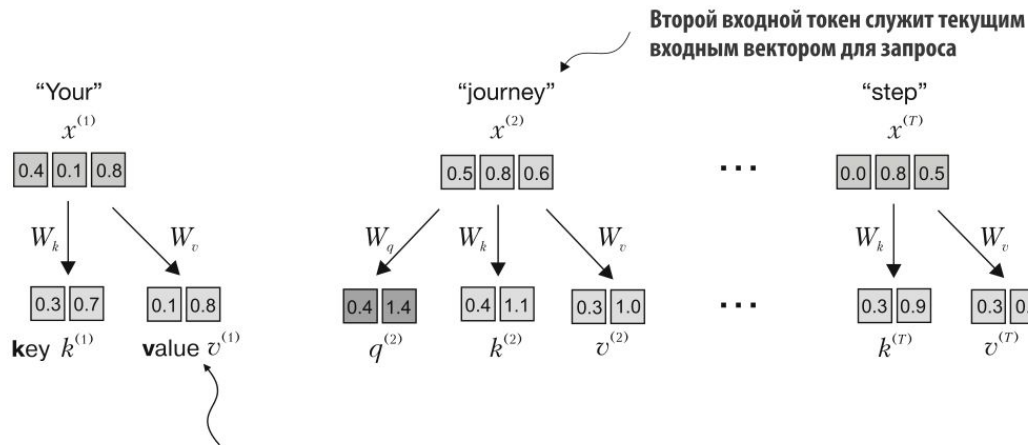
Три проекции одного эмбединга

- Вход — три числа, проекция — два



- context vector — выход внимания для токeна: смесь value с весами внимания
- веса внимания — attention scores после softmax; сумма по строке равна единице
- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю

Три проекции одного эмбединга

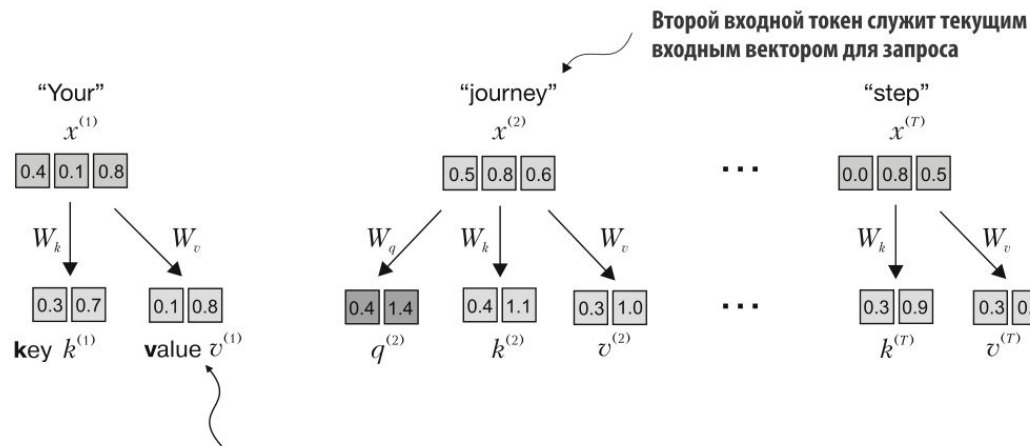


Это вектор значений, соответствующий первому входному токeну, полученный путем перемножения весовой матрицы W_v и входного токeна $x^{(1)}$

- Вход — три числа, проекция — два
- query ищет, key объявляет, value отдаёт

- context vector — выход внимания для токeна: смесь value с весами внимания
- веса внимания — attention scores после softmax; сумма по строке равна единице
- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю

Три проекции одного эмбединга



Второй входной токeн служит текущим входным вектором для запроса

Это вектор значений, соответствующий первому входному токeну, полученный путем перемножения весовой матрицы W_v и входного токeна $x^{(1)}$

- Вход — три числа, проекция — два
- query ищет, key объявляет, value отдаёт
- Совпадение — число, а не да/нет

- context vector — выход внимания для токeна: смесь value с весами внимания
- веса внимания — attention scores после softmax; сумма по строке равна единице
- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю

Размерности и три матрицы

$d = 3$ на входе, $d_{\text{out}} = 2$ на выходе

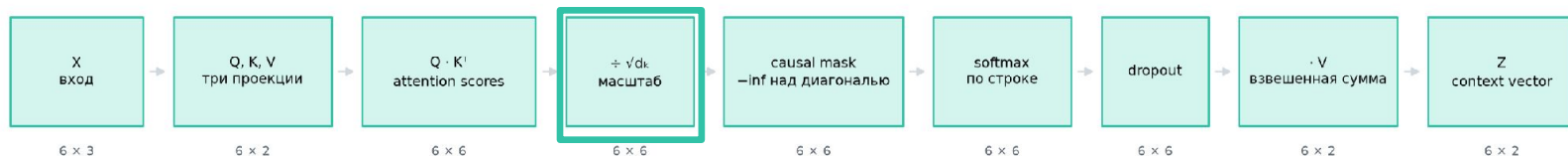
```
1 x_2 = inputs[1]          # второй токен — запрос
2 d_in = inputs.shape[1]   # размерность входного эмбединга, d = 3
3 d_out = 2 # размерность выхода; в GPT d_in = d_out, здесь разные для наглядности
4 torch.manual_seed(123)
5 W_query = torch.nn.Parameter(torch.rand(d_in, d_out), requires_grad=False)
6 W_key   = torch.nn.Parameter(torch.rand(d_in, d_out), requires_grad=False)
7 W_value = torch.nn.Parameter(torch.rand(d_in, d_out), requires_grad=False)
```

query второго токена, key и value всех

Один токен — запрос; ключи и значения нужны от всех

```
1 query_2 = x_2 @ W_query
2 key_2 = x_2 @ W_key
3 value_2 = x_2 @ W_value
4 print(query_2)      # tensor([0.4306, 1.4551])
5 keys = inputs @ W_key
6 values = inputs @ W_value
7 print("keys.shape:", keys.shape)    # torch.Size([6, 2])
8 print("values.shape:", values.shape) # torch.Size([6, 2])
```

Тот же расчёт, но с масштабом



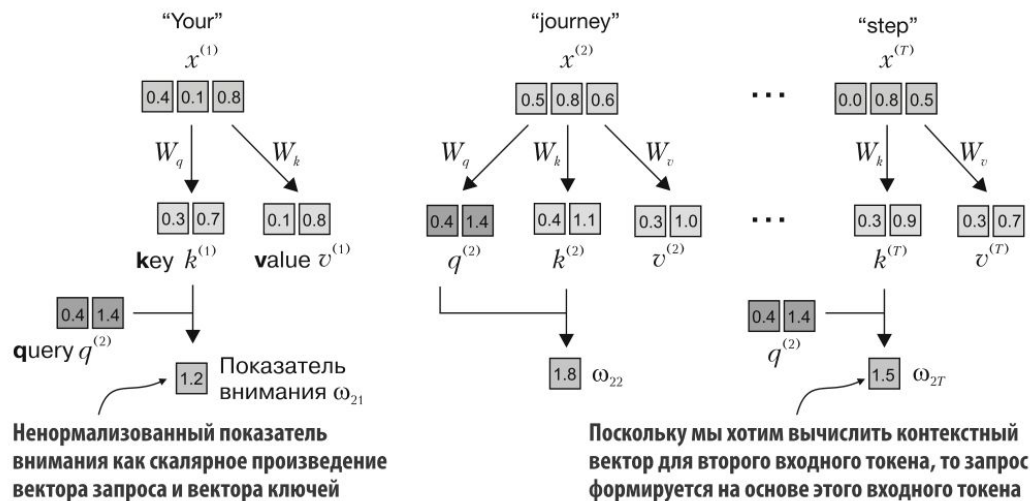
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Q, K, V — матрицы query, key, value; d_k — ширина вектора key

- context vector — выход внимания для токена: смесь value с весами внимания
- веса внимания — attention scores после softmax; сумма по строке равна единице
- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю

Источник: скрипт `figures/src/1_3_21.py`

Score — скалярное произведение q и k



- query второго токена — ко всем key
- $1,2 \cdot 1,8 \cdot 1,5$ — три показателя
- В коде те же числа: 1,8524

$$e_{ij} = \langle q_i, k_j \rangle = \langle W_q x_i, W_k x_j \rangle$$

$e(i, j)$ — attention score; q и k — query и key; W_q и W_k — матрицы, из которых они получены

- веса внимания — attention scores после softmax; сумма по строке равна единице
- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
- attention scores — числа до нормировки: скалярные произведения query и key

Score и деление на корень из d_k

Скалярное произведение q и k, затем масштаб

```
1 keys_2 = keys[1]          # ключ второго токена (индексация с нуля)
2 attn_score_22 = query_2.dot(keys_2)
3 print(attn_score_22)      # tensor(1.8524)
4
5 attn_scores_2 = query_2 @ keys.T # все показатели для запроса q(2) одним умножением
6 print(attn_scores_2)
7 # tensor([1.2705, 1.8524, 1.8111, 1.0795, 0.5577, 1.5440])
8 d_k = keys.shape[-1]      # размерность ключа, здесь 2
9 attn_weights_2 = torch.softmax(attn_scores_2 / d_k**0.5, dim=-1)
10 print(attn_weights_2)
11 # tensor([0.1500, 0.2264, 0.2199, 0.1311, 0.0906, 0.1820])
```

При $d_k = 2$ деление почти ничего не меняет: ставим его, чтобы формула совпала с рабочей. Эффект увидим на 64 и 512.

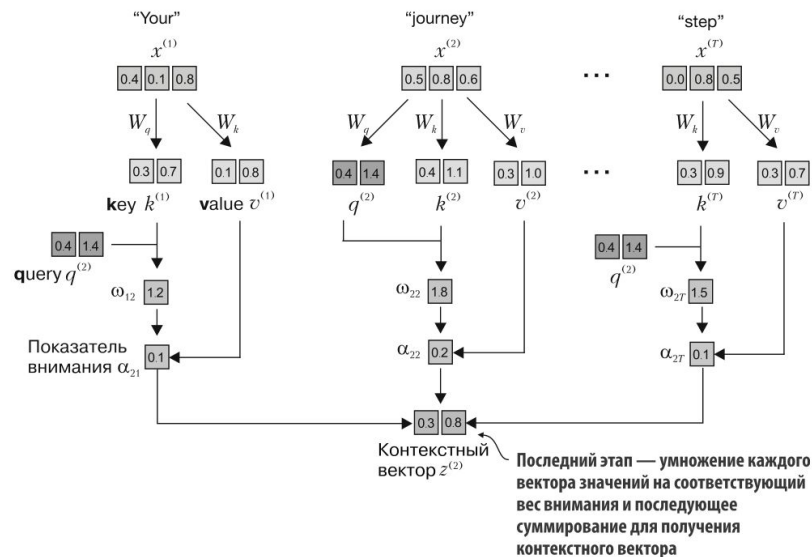
Score и деление на корень из d_k

Скалярное произведение q и k, затем масштаб

```
1 keys_2 = keys[1]          # ключ второго токена (индексация с нуля)
2 attn_score_22 = query_2.dot(keys_2)
3 print(attn_score_22)      # tensor(1.8524)
4
5 attn_scores_2 = query_2 @ keys.T # все показатели для запроса q(2) одним умножением
6 print(attn_scores_2)
7 # tensor([1.2705, 1.8524, 1.8111, 1.0795, 0.5577, 1.5440])
8 d_k = keys.shape[-1]      # размерность ключа, здесь 2
9 attn_weights_2 = torch.softmax(attn_scores_2 / d_k**0.5, dim=-1)
10 print(attn_weights_2)
11 # tensor([0.1500, 0.2264, 0.2199, 0.1311, 0.0906, 0.1820])
```

При $d_k = 2$ деление почти ничего не меняет: ставим его, чтобы формула совпала с рабочей. Эффект увидим на 64 и 512.

Context vector — смесь векторов value



- Показатели ω (омега) $\rightarrow$ softmax $\rightarrow$ веса α (альфа)
- Каждое value умножается на свой вес
- $z(2) = [0,3061; 0,8210]$

• веса внимания — attention scores после softmax; сумма по строке равна единице
query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
attention scores — числа до нормировки: скалярные произведения query и key

Context vector и несимметричность score

Веса умножаются на value; взгляд направлен

```
1 context_vec_2 = attn_weights_2 @ values
2 print(context_vec_2)          # tensor([0.3061, 0.8210])
3 queries = inputs @ W_query
4 scores = queries @ keys.T # матрица показателей  $L \times L$  для всех запросов
5 print(scores[1, 4].item(), scores[4, 1].item()) # journey->one, one->journey
6 print(torch.allclose(scores, scores.T))        # False
```

Насыщенный softmax не даёт градиента

$$\frac{\partial p_i}{\partial z_j} = p_i(\delta_{ij} - p_j)$$

p — веса после softmax; z — показатели до него; $\delta(i, j)$ — единица при $i = j$ и ноль иначе

- Показатели растут вместе с размерностью key — softmax становится ступенькой
- У ступеньки p_i равно нулю или единице, и производная обнуляется
- Нулевая производная — нулевой градиент: обучение не стартует

● веса внимания — attention scores после softmax; сумма по строке равна единице
query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
attention scores — числа до нормировки: скалярные произведения query и key

Откуда берётся корень из d_k

Дисперсия скалярного произведения = d_k , разброс = корень из d_k

величина	чему равна
координаты query и key	среднее 0, дисперсия 1 — допущение вывода
одно слагаемое $q_i \cdot k_i$	дисперсия 1
сумма из d_k слагаемых	дисперсия d_k
разброс показателей	корень из d_k
делитель в формуле	корень из d_k

Скалярное произведение независимых случайных величин:

$$q \cdot k = \sum q_i k_i$$

Запомнить:

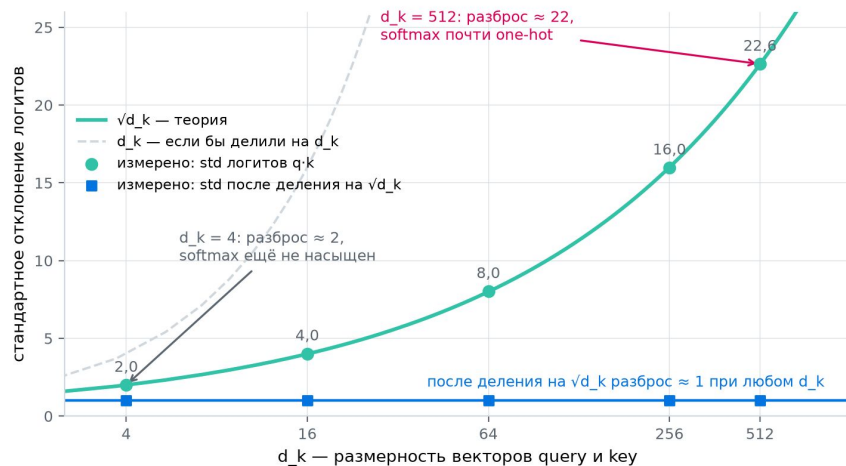
Дисперсии независимых слагаемых складываются, поэтому дисперсия суммы равна d_k , а стандартное отклонение — корню из d_k .

Опыт: десять тысяч пар векторов

```
1 torch.manual_seed(0)
2 for d_k in (4, 64, 512):
3     q = torch.randn(10_000, d_k)          # координаты из N(0, 1)
4     k = torch.randn(10_000, d_k)
5     s = (q * k).sum(dim=1)                 # 10 000 скалярных произведений
6     print(d_k, round(s.std().item(), 2), round((s / d_k**0.5).std().item(), 2))
7 # 4   1.99 1.0
8 # 64  8.01 1.0
9 # 512 22.49 0.99
```

Измеренный разброс лёг на корень

По вертикали — разброс attention scores, они же логиты



- Точки легли на кривую корня
- Пунктир d_k — рост, которого нет
- После деления — единица всегда

$$\sigma(e) = \sqrt{d_k} \Rightarrow e / \sqrt{d_k} \text{ — разброс снова } 1$$

σ (сигма) — разброс показателей; e — сами показатели; d_k — ширина вектора key

- веса внимания — attention scores после softmax; сумма по строке равна единице
- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
- attention scores — числа до нормировки: скалярные произведения query и key

Источник: Рис. 1.3.10 · своя схема, скрипт figures/src/1_3_10.py

Насыщение softmax и его производная

Что именно ломается без деления

```
1 z = torch.tensor([8., 0., -8.]) # разброс как при d_k = 64 без деления
2 p = torch.softmax(z, dim=0)
3 print(p)                        # tensor([0.9997, 0.0003, 0.0000])
4
5 z_scaled = z / 8 # те же показатели после деления на sqrt(64)
6 p_scaled = torch.softmax(z_scaled, dim=0)
7 print(p_scaled)                 # tensor([0.6652, 0.2447, 0.0900])
8 J = torch.diag(p) - torch.outer(p, p) # производная softmax по показателям
9 print(J.abs().max().item()) # 0.00034 — все элементы почти нули
10
11 J_scaled = torch.diag(p_scaled) - torch.outer(p_scaled, p_scaled)
12 print(J_scaled.abs().max().item()) # 0.2227 — в 660 раз больше
```

Без деления softmax почти one-hot: 0,9997 против 0,6652 на первом токене

Насыщение softmax и его производная

Что именно ломается без деления

```
1  z = torch.tensor([8., 0., -8.]) # разброс как при d_k = 64 без деления
2  p = torch.softmax(z, dim=0)
3  print(p)                        # tensor([0.9997, 0.0003, 0.0000])
4
5  z_scaled = z / 8 # те же показатели после деления на sqrt(64)
6  p_scaled = torch.softmax(z_scaled, dim=0)
7  print(p_scaled)                 # tensor([0.6652, 0.2447, 0.0900])
8  J = torch.diag(p) - torch.outer(p, p) # производная softmax по показателям
9  print(J.abs().max().item()) # 0.00034 — все элементы почти нули
10
11 J_scaled = torch.diag(p_scaled) - torch.outer(p_scaled, p_scaled)
12 print(J_scaled.abs().max().item()) # 0.2227 — в 660 раз больше
```

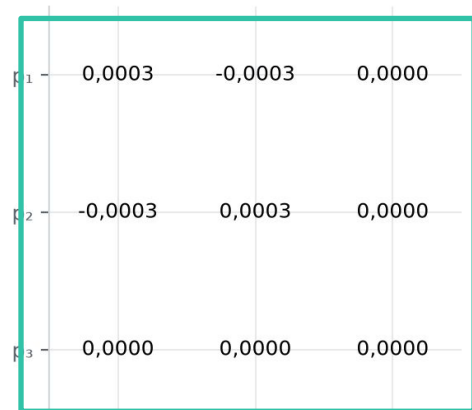
Без деления softmax почти one-hot: 0,9997 против 0,6652 на первом токене

Насыщение гасит все девять чисел

$\partial p(i)/\partial z(j) = p(i) \cdot (\delta(i,j) - p(j))$ — производная softmax

Матрица производных softmax: $\partial p_i / \partial z_j = p_i (\delta_{ij} - p_j)$

attention scores [8, 0, -8] — без деления



$p = [1,000, 0,000, 0,000]$ · максимум $|\partial p / \partial z| = 0,0003$

attention scores [1, 0, -1] — после деления на 8



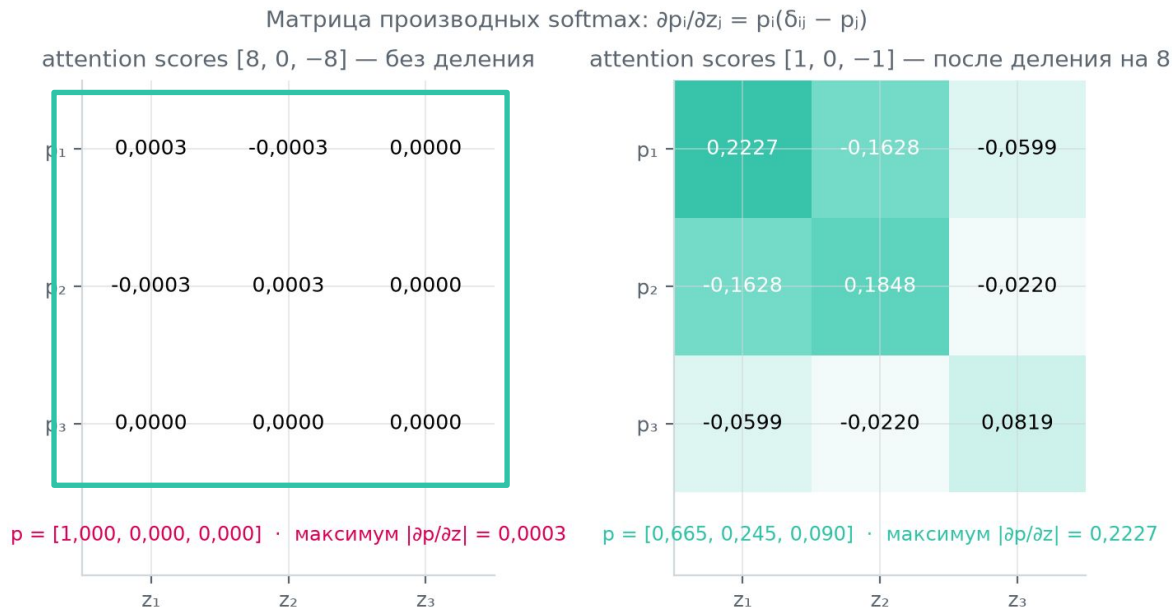
$p = [0,665, 0,245, 0,090]$ · максимум $|\partial p / \partial z| = 0,2227$

- веса внимания — attention scores после softmax; сумма по строке равна единице
- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
- attention scores — числа до нормировки: скалярные произведения query и key

Источник: Рис. 1.3.11 · своя схема, скрипт figures/src/1_3_11.py

Насыщение гасит все девять чисел

$\partial p(i)/\partial z(j) = p(i) \cdot (\delta(i,j) - p(j))$ — производная softmax



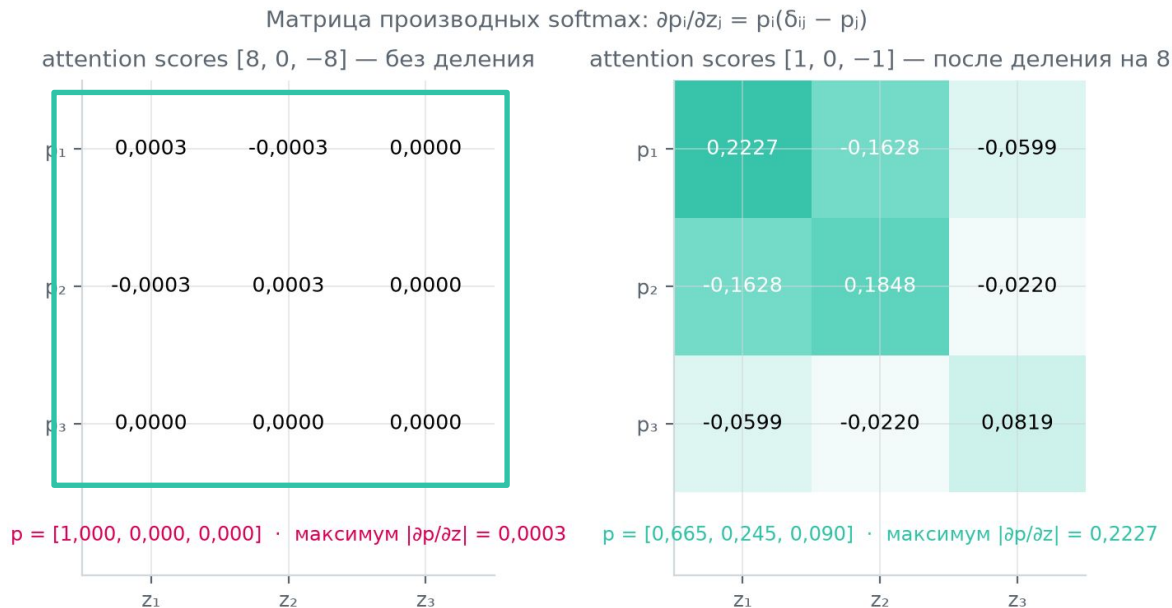
- Сигмоида на краях так же плоска

● веса внимания — attention scores после softmax; сумма по строке равна единице
query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
attention scores — числа до нормировки: скалярные произведения query и key

Источник: Рис. 1.3.11 · своя схема, скрипт figures/src/1_3_11.py

Насыщение гасит все девять чисел

$\partial p(i)/\partial z(j) = p(i) \cdot (\delta(i,j) - p(j))$ — производная softmax



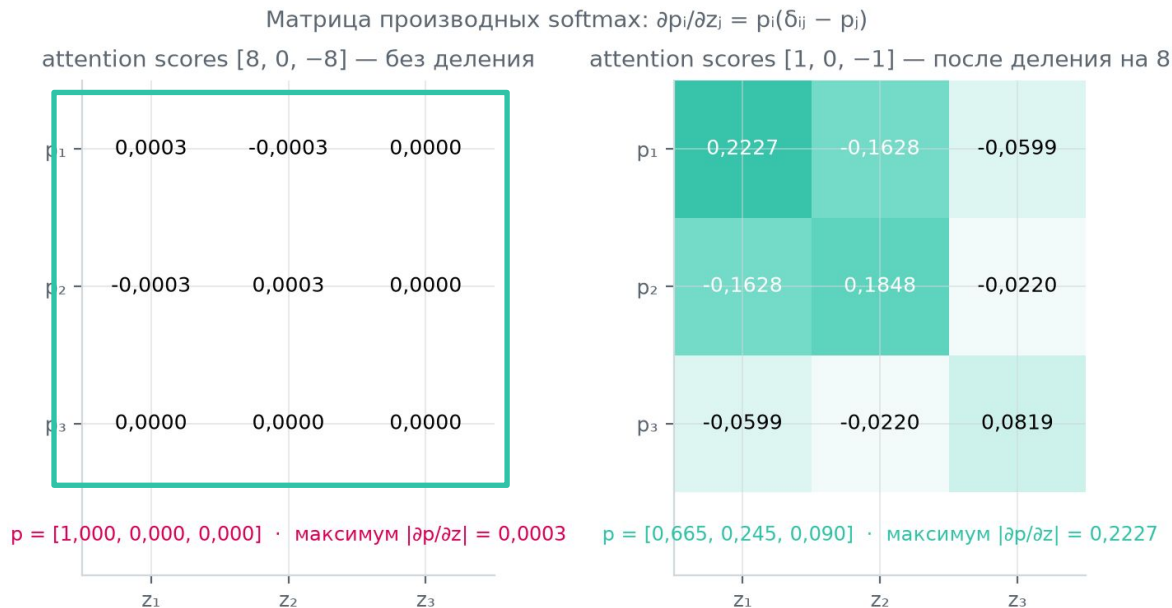
- Сигмоида на краях так же плоска
- Softmax гасит всю строку сразу

● веса внимания — attention scores после softmax; сумма по строке равна единице
query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
attention scores — числа до нормировки: скалярные произведения query и key

Источник: Рис. 1.3.11 · своя схема, скрипт figures/src/1_3_11.py

Насыщение гасит все девять чисел

$\partial p(i)/\partial z(j) = p(i) \cdot (\delta(i,j) - p(j))$ — производная softmax



- Сигмоида на краях так же плоска
- Softmax гасит всю строку сразу
- Градиент к Wq и Wk идёт здесь

● веса внимания — attention scores после softmax; сумма по строке равна единице
query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
attention scores — числа до нормировки: скалярные произведения query и key

Источник: Рис. 1.3.11 · своя схема, скрипт figures/src/1_3_11.py

Почему корень, а не сама d_k

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)V$$

Q, K, V — матрицы query, key, value; d_k — ширина вектора key

- Разброс показателей растёт как корень из d_k — делим ровно на него
- Поделить на сам d_k : при $d_k = 512$ разброс падает до 0,044, и softmax почти равномерный
- Не делить вовсе: softmax насыщается, производная обнуляется, обучение не стартует

Вторая типовая поломка: деление на корень забыли — формы и суммы строк этого не покажут.

- веса внимания — attention scores после softmax; сумма по строке равна единице
query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
attention scores — числа до нормировки: скалярные произведения query и key

Источник: Vaswani et al., 2017, формула (1), arXiv:1706.03762

SelfAttention_v1: конструктор

Класс наследует nn.Module — блока моделей PyTorch

```
1 import torch.nn as nn
2
3 class SelfAttention_v1(nn.Module):
4     def __init__(self, d_in, d_out):
5         super().__init__()
6         # три обучаемые матрицы проекций; nn.Parameter регистрирует их как параметры модуля,
7         # чтобы оптимизатор их видел и обновлял
8         self.W_query = nn.Parameter(torch.rand(d_in, d_out))
9         self.W_key = nn.Parameter(torch.rand(d_in, d_out))
10        self.W_value = nn.Parameter(torch.rand(d_in, d_out))
11
```

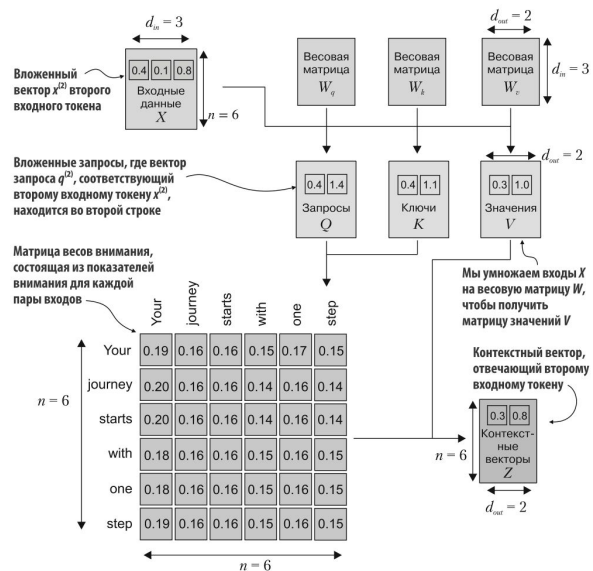
forward повторяет пройденные шаги

```
1
2 def forward(self, x):
3     keys = x @ self.W_key
4     queries = x @ self.W_query
5     values = x @ self.W_value
6     attn_scores = queries @ keys.T          # показатели внимания, L × L
7     attn_weights = torch.softmax(
8         attn_scores / keys.shape[-1]**0.5, dim=-1 # деление на sqrt(d_k), softmax по строкам
9     )
10    context_vec = attn_weights @ values      # взвешенная сумма значений
11    return context_vec
```

Шесть входов — шесть context vector

```
1 torch.manual_seed(123)
2 sa_v1 = SelfAttention_v1(d_in, d_out)
3 print(sa_v1(inputs))
4 # tensor([[0.2996, 0.8053],
5 #         [0.3061, 0.8210],
6 #         [0.3058, 0.8203],
7 #         [0.2948, 0.7939],
8 #         [0.2927, 0.7891],
9 #         [0.2990, 0.8040]], grad_fn=<MmBackward0>)
```

Все формы одной головы внимания



- Вход 6×3 , проекции Q, K, V — 6×2
- Матрица весов внимания всегда 6×6
- Ширину выхода задаёт только W_v

- веса внимания — attention scores после softmax; сумма по строке равна единице
- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
- attention scores — числа до нормировки: скалярные произведения query и key

v2: те же матрицы через nn.Linear

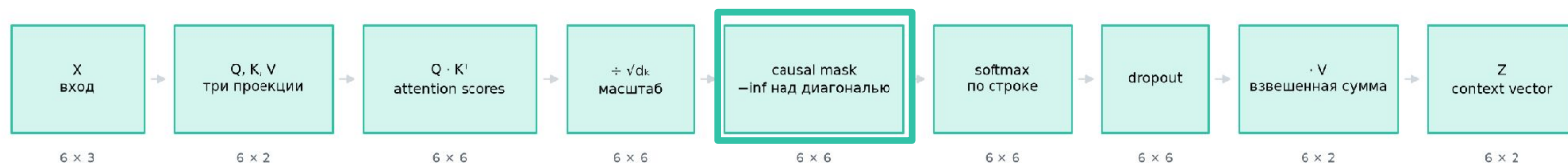
```
1 class SelfAttention_v2(nn.Module):
2     def __init__(self, d_in, d_out, qkv_bias=False):
3         super().__init__()
4         # nn.Linear без bias — то же умножение на матрицу,
5         # но инициализация рассчитана на устойчивое обучение
6         self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
7         self.W_key   = nn.Linear(d_in, d_out, bias=qkv_bias)
8         self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
9
```

v2: forward вызывает слои

```
1 def forward(self, x):
2     keys = self.W_key(x)
3     queries = self.W_query(x)
4     values = self.W_value(x)
5     attn_scores = queries @ keys.T
6     attn_weights = torch.softmax(
7         attn_scores / keys.shape[-1]**0.5, dim=-1
8     )
9     context_vec = attn_weights @ values
10    return context_vec
11
12 torch.manual_seed(789)
13 sa_v2 = SelfAttention_v2(d_in, d_out)
14 print(sa_v2(inputs))
```

Первая строка выхода — $[-0,0739; 0,0713]$: другая схема инициализации весов

Куда встаёт causal mask



$$a_{ij} = -\infty \Rightarrow e^{a_{ij}} = 0$$

$a(i, j)$ — показатель пары до softmax; e в степени — экспонента, она и даёт ноль

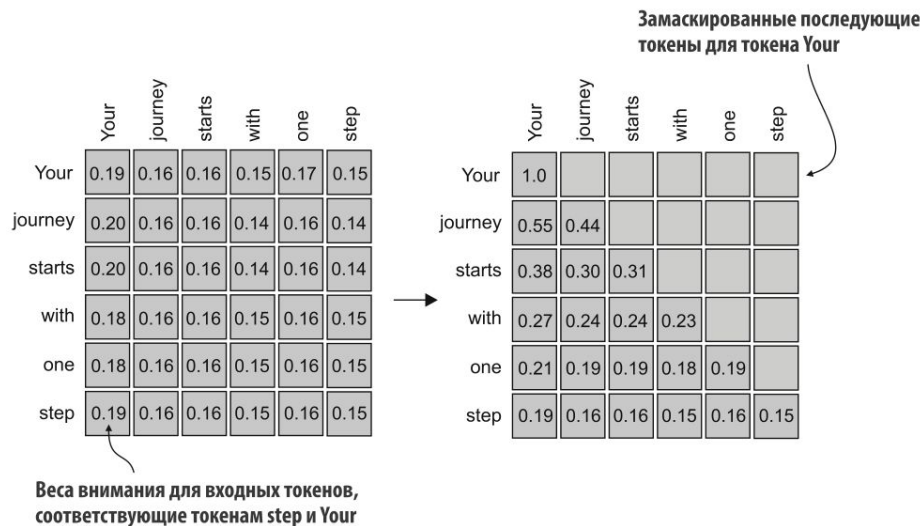
- Маска ставится ДО softmax: иначе строки перестают давать единицу

- веса внимания — attention scores после softmax; сумма по строке равна единице
- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
- attention scores — числа до нормировки: скалярные произведения query и key

План занятия

- 1 Что считает внимание
- 2 Внимание без параметров
- 3 Три матрицы: query, key, value
- 4 Causal mask и dropout**
- 5 Проверка и отладка
- 6 Позиция внутри внимания

Маска закрывает всё выше диагонали



- Слева все веса, справа — маска
- Строки нормируются заново

У journey два веса: 0,55 и 0,44.

- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
- attention scores — числа до нормировки: скалярные произведения query и key
- causal mask — запрет смотреть вперёд: $-\infty$ над диагональю до softmax

Веса внимания на тех же проекциях

query и key берём из готового объекта sa_v2 из раздела про компактный класс

```
1 queries = sa_v2.W_query(inputs)      # проекции из готового объекта, чтобы не заводить новые матрицы
2 keys   = sa_v2.W_key(inputs)
3 attn_scores = queries @ keys.T
4 attn_weights = torch.softmax(attn_scores / keys.shape[-1]**0.5, dim=-1)
5 print(attn_weights)
6 # tensor([[[0.1921, 0.1646, 0.1652, 0.1550, 0.1721, 0.1510],
7 #          [0.2041, 0.1659, 0.1662, 0.1496, 0.1665, 0.1477],
8 #          [0.2036, 0.1659, 0.1662, 0.1498, 0.1664, 0.1480],
9 #          [0.1869, 0.1667, 0.1668, 0.1571, 0.1661, 0.1564],
10 #          [0.1830, 0.1669, 0.1670, 0.1588, 0.1658, 0.1585],
11 #          [0.1935, 0.1663, 0.1666, 0.1542, 0.1666, 0.1529]]])
```

Нижний треугольник из единиц

Прямой способ: маску умножают на готовые веса

```
1 context_length = attn_scores.shape[0]
2 mask_simple = torch.tril(torch.ones(context_length, context_length))
3 print(mask_simple)
4 # tensor([[1., 0., 0., 0., 0., 0.],
5 #         [1., 1., 0., 0., 0., 0.],
6 #         [1., 1., 1., 0., 0., 0.],
7 #         [1., 1., 1., 1., 0., 0.],
8 #         [1., 1., 1., 1., 1., 0.],
9 #         [1., 1., 1., 1., 1., 1.]])
```

Занулили — суммы строк не единицы

Ошибку не видно ни по формам, ни по суммам весов

```
1 masked_simple = attn_weights * mask_simple
2 print(masked_simple)
3 # tensor([[[0.1921, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
4 #          [0.2041, 0.1659, 0.0000, 0.0000, 0.0000, 0.0000],
5 #          [0.2036, 0.1659, 0.1662, 0.0000, 0.0000, 0.0000],
6 #          [0.1869, 0.1667, 0.1668, 0.1571, 0.0000, 0.0000],
7 #          [0.1830, 0.1669, 0.1670, 0.1588, 0.1658, 0.0000],
8 #          [0.1935, 0.1663, 0.1666, 0.1542, 0.1666, 0.1529]]])
9
10 print(masked_simple.sum(dim=-1))
11 # tensor([0.1921, 0.3700, 0.5357, 0.6775, 0.8415, 1.0000])
```

Делим каждую строку на её сумму

Маска после softmax обязывает перенормировать строки

```
1 row_sums = masked_simple.sum(dim=-1, keepdim=True)
2 masked_simple_norm = masked_simple / row_sums
3 print(masked_simple_norm)    # строки: Your, journey, starts, with, one, step
4 # tensor([[1.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
5 #        [0.5517, 0.4483, 0.0000, 0.0000, 0.0000, 0.0000],
6 #        [0.3800, 0.3097, 0.3103, 0.0000, 0.0000, 0.0000],
7 #        [0.2758, 0.2460, 0.2462, 0.2319, 0.0000, 0.0000],
8 #        [0.2175, 0.1983, 0.1984, 0.1888, 0.1971, 0.0000],
9 #        [0.1935, 0.1663, 0.1666, 0.1542, 0.1666, 0.1529]])
```

Третья типовая поломка: маска поставлена после softmax — строки перестают давать единицу.

Делим каждую строку на её сумму

Маска после softmax обязывает перенормировать строки

```
1 row_sums = masked_simple.sum(dim=-1, keepdim=True)
2 masked_simple_norm = masked_simple / row_sums
3 print(masked_simple_norm)    # строки: Your, journey, starts, with, one, step
4 # tensor([[1.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
5 #         [0.5517, 0.4483, 0.0000, 0.0000, 0.0000, 0.0000],
6 #         [0.3800, 0.3097, 0.3103, 0.0000, 0.0000, 0.0000],
7 #         [0.2758, 0.2460, 0.2462, 0.2319, 0.0000, 0.0000],
8 #         [0.2175, 0.1983, 0.1984, 0.1888, 0.1971, 0.0000],
9 #         [0.1935, 0.1663, 0.1666, 0.1542, 0.1666, 0.1529]])
```

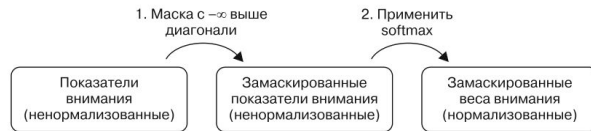
Третья типовая поломка: маска поставлена после softmax — строки перестают давать единицу.

triu отмечает всё над диагональю

Единицы строго над диагональю, сама диагональ остаётся открытой

```
1 mask = torch.triu(torch.ones(context_length, context_length), diagonal=1)
2 masked = attn_scores.masked_fill(mask.bool(), -torch.inf)
3 print(masked)
4 # tensor([[0.2899, -inf, -inf, -inf, -inf, -inf],
5 #         [0.4656, 0.1723, -inf, -inf, -inf, -inf],
6 #         [0.4594, 0.1703, 0.1731, -inf, -inf, -inf],
7 #         [0.2642, 0.1024, 0.1036, 0.0186, -inf, -inf],
8 #         [0.2183, 0.0874, 0.0882, 0.0177, 0.0786, -inf],
9 #         [0.3408, 0.1270, 0.1290, 0.0198, 0.1290, 0.0078]])
```

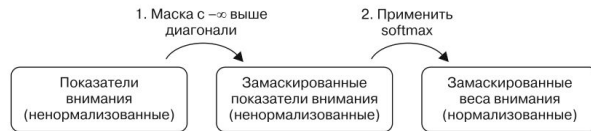
Softmax сам ставит нули на $-\infty$



```
1 attn_weights = torch.softmax(masked / keys.shape[-1]**0.5, dim=-1)
2 print(attn_weights)
3 # tensor([[[1.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
4 #          [0.5517, 0.4483, 0.0000, 0.0000, 0.0000, 0.0000],
5 #          [0.3800, 0.3097, 0.3103, 0.0000, 0.0000, 0.0000],
6 #          [0.2758, 0.2460, 0.2462, 0.2319, 0.0000, 0.0000],
7 #          [0.2175, 0.1983, 0.1984, 0.1888, 0.1971, 0.0000],
8 #          [0.1935, 0.1663, 0.1666, 0.1542, 0.1666, 0.1529]]]])
9
10 print(torch.allclose(attn_weights, masked_simple_norm, atol=1e-6)) # True
```

- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
- attention scores — числа до нормировки: скалярные произведения query и key
- causal mask — запрет смотреть вперёд: $-\infty$ над диагональю до softmax

Softmax сам ставит нули на $-\infty$

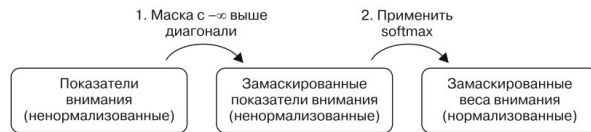


```
1 attn_weights = torch.softmax(masked / keys.shape[-1]**0.5, dim=-1)
2 print(attn_weights)
3 # tensor([[[1.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
4 #          [0.5517, 0.4483, 0.0000, 0.0000, 0.0000, 0.0000],
5 #          [0.3800, 0.3097, 0.3103, 0.0000, 0.0000, 0.0000],
6 #          [0.2758, 0.2460, 0.2462, 0.2319, 0.0000, 0.0000],
7 #          [0.2175, 0.1983, 0.1984, 0.1888, 0.1971, 0.0000],
8 #          [0.1935, 0.1663, 0.1666, 0.1542, 0.1666, 0.1529]]]])
9
10 print(torch.allclose(attn_weights, masked_simple_norm, atol=1e-6)) # True
```

показатели → маска $-\infty$ → деление на корень → softmax

- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
- attention scores — числа до нормировки: скалярные произведения query и key
- causal mask — запрет смотреть вперёд: $-\infty$ над диагональю до softmax

Softmax сам ставит нули на $-\infty$

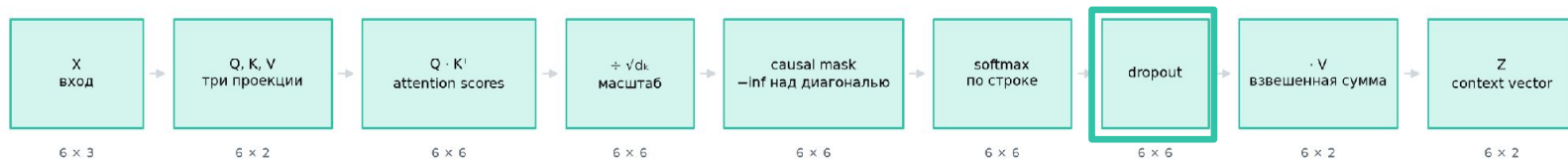


```
1 attn_weights = torch.softmax(masked / keys.shape[-1]**0.5, dim=-1)
2 print(attn_weights)
3 # tensor([[[1.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
4 #          [0.5517, 0.4483, 0.0000, 0.0000, 0.0000, 0.0000],
5 #          [0.3800, 0.3097, 0.3103, 0.0000, 0.0000, 0.0000],
6 #          [0.2758, 0.2460, 0.2462, 0.2319, 0.0000, 0.0000],
7 #          [0.2175, 0.1983, 0.1984, 0.1888, 0.1971, 0.0000],
8 #          [0.1935, 0.1663, 0.1666, 0.1542, 0.1666, 0.1529]]]])
9
10 print(torch.allclose(attn_weights, masked_simple_norm, atol=1e-6)) # True
```

Маска — единственное архитектурное различие энкодера и декодера

- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
- attention scores — числа до нормировки: скалярные произведения query и key
- causal mask — запрет смотреть вперёд: $-\infty$ над диагональю до softmax

Где в механизме стоит dropout

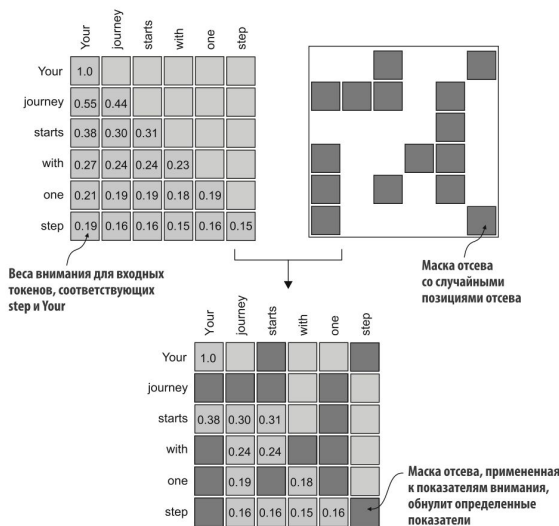


- Dropout ставят на веса внимания — сразу после softmax
- Реже — после умножения весов на value

- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
- attention scores — числа до нормировки: скалярные произведения query и key
- causal mask — запрет смотреть вперёд: $-\text{inf}$ над диагональю до softmax

Источник: скрипт `figures/src/1_3_21.py`

Поверх маски — случайная маска



- Dropout работает только на обучении
- Ставим его на веса внимания

Доля на занятии — 0,5; у GPT берут 0,1 или 0,2.

- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
- attention scores — числа до нормировки: скалярные произведения query и key
- causal mask — запрет смотреть вперёд: $-\infty$ над диагональю до softmax

Половина нулей, остальные — двойки

Сначала смотрим, что dropout делает с матрицей из единиц

```
1 torch.manual_seed(123)
2 dropout = torch.nn.Dropout(0.5)      # доля обнуляемых элементов — 50 %
3 example = torch.ones(6, 6)
4 print(dropout(example))
5 # tensor([[2., 2., 0., 2., 2., 0.],
6 #         [0., 0., 0., 2., 0., 2.],
7 #         [2., 2., 2., 2., 0., 2.],
8 #         [0., 2., 2., 0., 0., 2.],
9 #         [0., 2., 0., 2., 0., 2.],
10 #         [0., 2., 2., 2., 2., 0.]])
```

Dropout на весах и вход-батч

После dropout суммы строк единице не равны — и не должны

```
1 torch.manual_seed(123)
2 print(dropout(attn_weights))
3 # tensor([[[2.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
4 #          [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
5 #          [0.7599, 0.6194, 0.6206, 0.0000, 0.0000, 0.0000],
6 #          [0.0000, 0.4921, 0.4925, 0.0000, 0.0000, 0.0000],
7 #          [0.0000, 0.3966, 0.0000, 0.3775, 0.0000, 0.0000],
8 #          [0.0000, 0.3327, 0.3331, 0.3084, 0.3331, 0.0000]])])
9
10 batch = torch.stack((inputs, inputs), dim=0) # два текста по шесть токенов, эмбединг из трёх чисел
11 print(batch.shape)                          # torch.Size([2, 6, 3])
```

Маска живёт в буфере, не в весах

Конструктор нового класса: три проекции, dropout и маска-буфер

```
1 class CausalAttention(nn.Module):
2     def __init__(self, d_in, d_out, context_length,
3         dropout, qkv_bias=False):
4         super().__init__()
5         self.d_out = d_out
6         self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
7         self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
8         self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
9         self.dropout = nn.Dropout(dropout) # новое по сравнению с SelfAttention: слой dropout
10        # маска — не параметр, а буфер: она не обучается, но должна переезжать на устройство вместе с модулем
11        self.register_buffer(
12            'mask',
13            torch.triu(torch.ones(context_length, context_length), diagonal=1)
14        )
```

Маска живёт в буфере, не в весах

Конструктор нового класса: три проекции, dropout и маска-буфер

```
1 class CausalAttention(nn.Module):
2     def __init__(self, d_in, d_out, context_length,
3         dropout, qkv_bias=False):
4         super().__init__()
5         self.d_out = d_out
6         self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
7         self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
8         self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
9         self.dropout = nn.Dropout(dropout) # новое по сравнению с SelfAttention: слой dropout
10        # маска — не параметр, а буфер: она не обучается, но должна переезжать на устройство вместе с модулем
11        self.register_buffer(
12            'mask',
13            torch.triu(torch.ones(context_length, context_length), diagonal=1)
14        )
```

Ось батча проходит сквозь умножения

Прямой проход: ось батча остаётся первой, маска — по числу токенов

```
1 def forward(self, x):
2     b, num_tokens, d_in = x.shape          # батч, токены, ширина эмбединга
3     keys = self.W_key(x)
4     queries = self.W_query(x)
5     values = self.W_value(x)
6     # transpose(1, 2) меняет оси токенов и признаков, ось батча остаётся первой
7     attn_scores = queries @ keys.transpose(1, 2)
8     # маска обрезается до числа токенов, правка идёт на месте — с подчёркиванием
9     attn_scores.masked_fill_(
10         self.mask.bool()[:num_tokens, :num_tokens], -torch.inf)
11     attn_weights = torch.softmax(attn_scores / keys.shape[-1]**0.5, dim=-1)
12     attn_weights = self.dropout(attn_weights)
13     context_vec = attn_weights @ values
14     return context_vec
```

Обучаются три матрицы, маска — нет

Модуль с нулевым dropout на батче из двух текстов

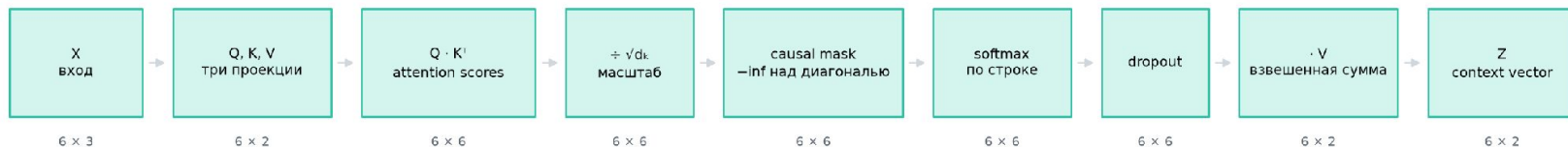
```
1 torch.manual_seed(123)
2 context_length = batch.shape[1]
3 ca = CausalAttention(d_in, d_out, context_length, 0.0) # без dropout
4 context_vecs = ca(batch)
5 print("context_vecs.shape:", context_vecs.shape) # torch.Size([2, 6, 2])
6 print([name for name, _ in ca.named_parameters()])
7 # ['W_query.weight', 'W_key.weight', 'W_value.weight']
8 print([name for name, _ in ca.named_buffers()]) # ['mask']
```

Перенормировка после обнуления даёт те же веса, что и маска $-\infty$: пропорции сохраняются, знаменатель заменяется.

План занятия

- 1 Что считает внимание
- 2 Внимание без параметров
- 3 Три матрицы: query, key, value
- 4 Causal mask и dropout
- 5 Проверка и отладка**
- 6 Позиция внутри внимания

Голова внимания собрана целиком



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

M — матрица маски: ноль на разрешённых парах и минус бесконечность на запрещённых

- M — causal mask: нули ниже диагонали, минус бесконечность выше
- Далее сверяем свою реализацию с готовой из PyTorch

- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
- attention scores — числа до нормировки: скалярные произведения query и key
- causal mask — запрет смотреть вперёд: -inf над диагональю до softmax

Источник: скрипт `figures/src/1_3_21.py`

Сверка с эталоном до 1e-6

Четыре строки автотеста: та же формула, те же проекции

```
1 import torch.nn.functional as F
2
3 q = ca.W_query(batch)          # проекции из нашего модуля
4 k = ca.W_key(batch)
5 v = ca.W_value(batch)
6 ref = F.scaled_dot_product_attention(q, k, v, is_causal=True)
7
8 print((ref - context_vecs).abs().max().item())      # 0.0
9 print(torch.allclose(ref, context_vecs, atol=1e-6)) # True
```

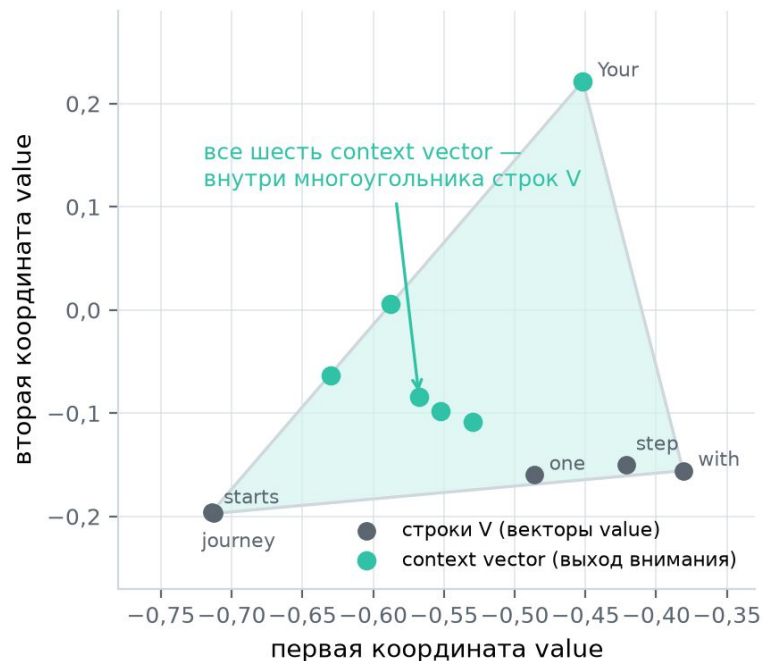
Свой класс нужен там, где формулу меняют: маски, головы, кэш. Эталон делает ровно базовую формулу — поэтому и годится как тест.

Выход не выходит за диапазон value

Каждая строка выхода — смесь строк value с весами внимания

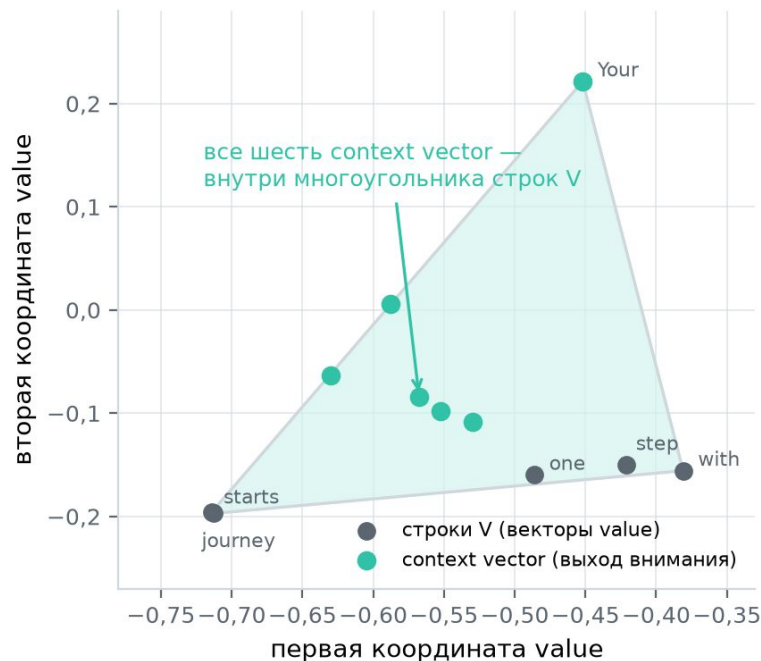
```
1 lo = v[0].min(dim=0).values      # покоординатный минимум по строкам V
2 hi = v[0].max(dim=0).values      # покоординатный максимум
3 print(lo, hi) # tensor([-0.7142, -0.1971]) tensor([-0.3809, 0.2216])
4 inside = (context_vecs[0] >= lo - 1e-6) & (context_vecs[0] <= hi + 1e-6)
5 print(inside.all().item())       # True
```

Внимание смешивает, но не создаёт



- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
- attention scores — числа до нормировки: скалярные произведения query и key
- causal mask — запрет смотреть вперёд: $-\infty$ над диагональю до softmax

Внимание смешивает, но не создаёт

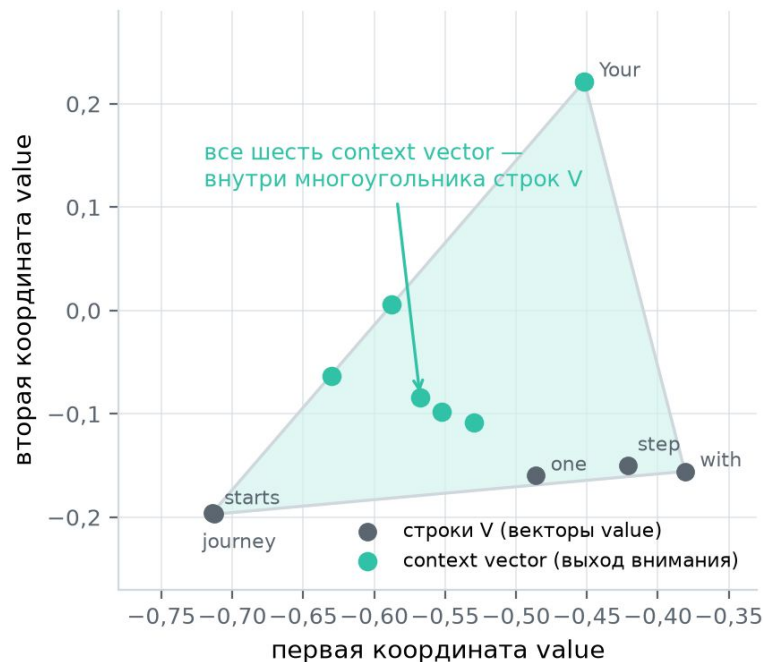


- Зелёные точки —
внутри оболочки

● query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
attention scores — числа до нормировки: скалярные произведения query и key
causal mask — запрет смотреть вперёд: $-\infty$ над диагональю до softmax

Источник: Рис. 1.3.16 · своя схема, figures/src/1_3_16.py

Внимание смешивает, но не создаёт



- Зелёные точки — внутри оболочки
- Нелинейность — только в весах

● query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
attention scores — числа до нормировки: скалярные произведения query и key
causal mask — запрет смотреть вперёд: $-\infty$ над диагональю до softmax

Источник: Рис. 1.3.16 · своя схема, figures/src/1_3_16.py

Три способа сломать внимание

Формы у всех трёх верные — расходятся только числа

что сломано	что показывают формы	чем ловится
softmax по столбцам ($\text{dim} = 0$)	те же 6×6 , исключения нет	суммы строк: 0,187...2,407 вместо единиц
забыли деление на корень из d_k	формы верные, суммы строк единичные	только сверка с эталоном: 0,0029 при $d_k = 2$
маска после softmax	формы верные	суммы строк меньше единицы

Код всех трёх поломок — в ноутбуке занятия 4.

Порядок отладки внимания

- Формы тензоров
- Суммы строк весов — единицы
- Деление на корень до softmax
- $-\infty$ до softmax, диагональ открыта
- Сверка с эталоном, допуск $1e-6$

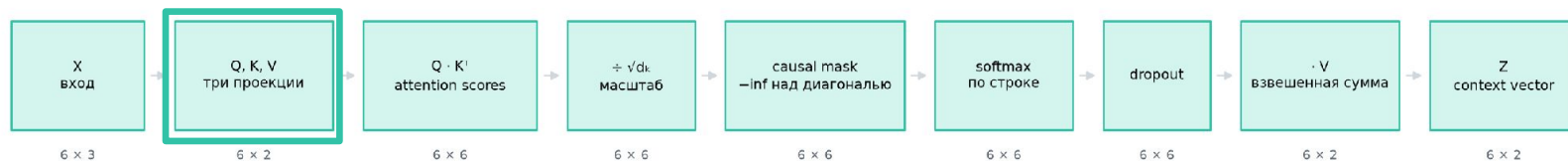
Насколько велик разброс в реальных моделях — вопрос замера, а не вывода: в уроке измерены только учебные размерности.

План занятия

- 1 Что считает внимание
- 2 Внимание без параметров
- 3 Три матрицы: query, key, value
- 4 Causal mask и dropout
- 5 Проверка и отладка
- 6 Позиция внутри внимания**

Позиция входит в сравнение токенов

На занятии 2 номер позиции вшивался в сам вектор токена



- Занятие 2: к каждому эмбеддингу прибавляли вектор позиции — рамка на схеме
- Позиция оставалась абсолютной: номер вшит в сам вектор токена
- Сегодня вносим её туда, где токены сравниваются, — и видим расстояние, а не номера

- query / key / value — три линейные проекции эмбеддинга: что ищу, чем себя объявляю, что отдаю
- attention scores — числа до нормировки: скалярные произведения query и key
- causal mask — запрет смотреть вперёд: $-\text{inf}$ над диагональю до softmax

Источник: скрипт `figures/src/1_3_21.py`

Позицию прибавляли к эмбедингу

Три строки прошлого занятия — на наших шести токенах

```
1 context_length, output_dim = 6, 3      # длина нашей фразы и ширина эмбединга
2 token_embeddings = inputs               # наши шесть векторов по три числа
3
4 pos_embedding_layer = torch.nn.Embedding(context_length, output_dim) # таблица
5 pos_embeddings = pos_embedding_layer(torch.arange(context_length)) # позиции 0...5
6 input_embeddings = token_embeddings + pos_embeddings          # вход модели
7
8 print(input_embeddings.shape)           # torch.Size([6, 3]) — форма та же, что X
```

Прибавленная позиция утекает в score

Расстояние одно, а числа отличаются в десять раз

```
1 q = torch.tensor([1.0, 2.0])    # query первого токена пары
2 k = torch.tensor([3.0, 0.5])    # key второго токена пары
3 p = torch.tensor([0.3, -0.4])   # вектор позиции: один шаг
4
5 print((q + 1 * p) @ (k + 3 * p)) # позиции 1 и 3: tensor(3.9500)
6 print((q + 11 * p) @ (k + 13 * p)) # позиции 11 и 13: tensor(40.9500)
```

Расстояние в обеих парах равно двум, а score отличается в десять раз.

Внимание видит только разность позиций

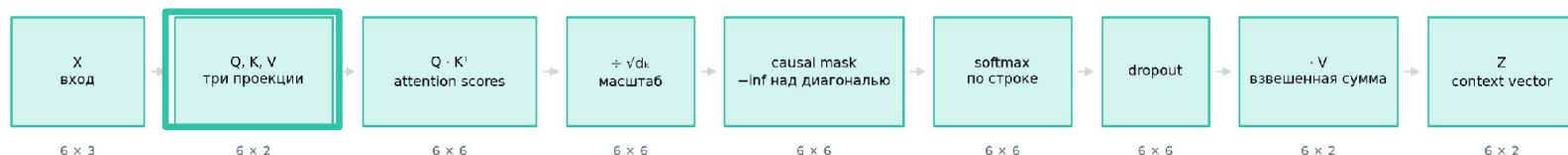
$$\langle f(x, i), f(y, j) \rangle = g(x, y, i - j)$$

f — искомое преобразование вектора с учётом позиции; g — результат, зависящий только от разности позиций

- Слева — скалярное произведение двух векторов с внесённой позицией
- Справа — результат зависит от $i - j$, то есть от расстояния, а не от номеров
- Прибавленный к эмбедингу вектор позиции этого не даёт: он несёт номер

- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
- attention scores — числа до нормировки: скалярные произведения query и key
- causal mask — запрет смотреть вперёд: $-\infty$ над диагональю до softmax

Позиция входит в сравнение токенов



$$\text{score} = \langle F(i) q, G(j) k \rangle = q^\top A(i, j) k$$

F и G — преобразования query и key по их позициям; A — матрица, в которую они сворачиваются

- Меняем не готовый score, а сами векторы query и key
- F и G — обозначения преобразований; если оба линейны, всё сводится к одной матрице A

- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
- attention scores — числа до нормировки: скалярные произведения query и key
- causal mask — запрет смотреть вперёд: $-\infty$ над диагональю до softmax

Источник: скрипт `figures/src/1_3_21.py`

Поворот не меняет произведения

Повернули оба вектора на один угол — score прежний

```
1 def rotate(v, angle):
2     """Поворот двумерного вектора на угол angle."""
3     angle = torch.tensor(angle)
4     c, s = torch.cos(angle), torch.sin(angle)
5     R = torch.stack([torch.stack([c, -s]), torch.stack([s, c])]) # матрица поворота
6     return R @ v
7
8 q = torch.tensor([1.0, 2.0])
9 k = torch.tensor([3.0, 0.5])
10 print(q @ k) # tensor(4.)
11 print(rotate(q, 0.7) @ rotate(k, 0.7)) # tensor(4.) — совместный поворот ничего не изменил
```

Поворот не меняет произведения

Повернули оба вектора на один угол — score прежний

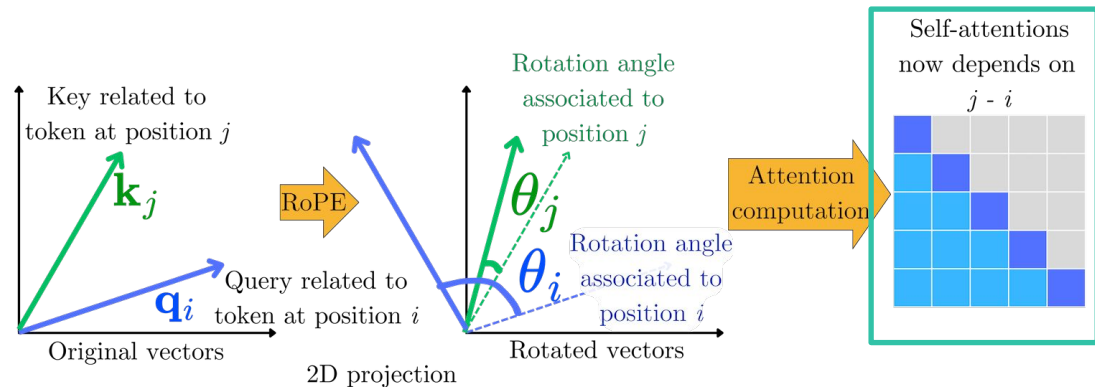
```
1 def rotate(v, angle):
2     """Поворот двумерного вектора на угол angle."""
3     angle = torch.tensor(angle)
4     c, s = torch.cos(angle), torch.sin(angle)
5     R = torch.stack([torch.stack([c, -s]), torch.stack([s, c])]) # матрица поворота
6     return R @ v
7
8 q = torch.tensor([1.0, 2.0])
9 k = torch.tensor([3.0, 0.5])
10 print(q @ k) # tensor(4.)
11 print(rotate(q, 0.7) @ rotate(k, 0.7)) # tensor(4.) — совместный поворот ничего не изменил
```

Угол по позиции: остаётся расстояние

Пары позиций (1, 3) и (11, 13) дают одно и то же число

```
1 | theta = 0.1                                # шаг угла на одну позицию
2 | print(rotate(q, theta*1) @ rotate(k, theta*3)) # позиции 1 и 3: tensor(5.0129)
3 | print(rotate(q, theta*11) @ rotate(k, theta*13)) # позиции 11 и 13: tensor(5.0129)
```

Поворот кодирует позицию



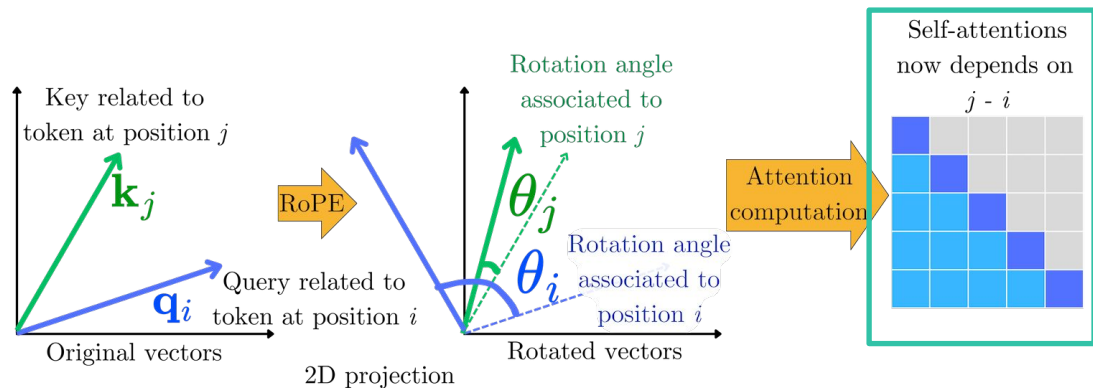
- Слева — query и key до поворота

$$\langle R(\theta_i) q, R(\theta_j) k \rangle = g(q, k, j - i)$$

$R(\theta)$ — поворот на угол θ (тета); q и k — query и key; $j - i$ — расстояние

- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
attention scores — числа до нормировки: скалярные произведения query и key
causal mask — запрет смотреть вперёд: $-\infty$ над диагональю до softmax

Поворот кодирует позицию



- Слева — query и key до поворота
- Справа — вес зависит от разности $j - i$

$$\langle R(\theta_i) q, R(\theta_j) k \rangle = g(q, k, j - i)$$

$R(\theta)$ — поворот на угол θ (тета); q и k — query и key; $j - i$ — расстояние

- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
- attention scores — числа до нормировки: скалярные произведения query и key
- causal mask — запрет смотреть вперёд: $-\infty$ над диагональю до softmax

Поворот для настоящей размерности

Своя частота у каждой пары координат

```
1 def compute_rope_params(dim, theta_base=10_000, context_length=4096):
2     # свои частоты для каждой пары координат — как в синусоидальной формуле
3     inv_freq = 1.0 / (theta_base ** (torch.arange(0, dim, 2).float() / dim))
4     positions = torch.arange(context_length, dtype=torch.float32)
5     angles = positions[:, None] * inv_freq[None, :] # угол = позиция x частота
6     angles = torch.cat([angles, angles], dim=1) # угол нужен обеим половинам вектора
7     return torch.cos(angles), torch.sin(angles)
8
9 def apply_rope(x, cos, sin):
10    # x: (seq_len, dim); первая половина координат играет роль «x», вторая — «y»
11    seq_len, dim = x.shape
12    x1, x2 = x[:, :dim // 2], x[:, dim // 2:]
13    rotated = torch.cat((-x2, x1), dim=-1) # поворот пары: (x, y) -> (-y, x)
14    return x * cos[:seq_len] + rotated * sin[:seq_len] # классическая формула поворота
```

Поворот для настоящей размерности

Своя частота у каждой пары координат

```
1 def compute_rope_params(dim, theta_base=10_000, context_length=4096):
2     # свои частоты для каждой пары координат — как в синусоидальной формуле
3     inv_freq = 1.0 / (theta_base ** (torch.arange(0, dim, 2).float() / dim))
4     positions = torch.arange(context_length, dtype=torch.float32)
5     angles = positions[:, None] * inv_freq[None, :] # угол = позиция x частота
6     angles = torch.cat([angles, angles], dim=1) # угол нужен обеим половинам вектора
7     return torch.cos(angles), torch.sin(angles)
8
9 def apply_rope(x, cos, sin):
10    # x: (seq_len, dim); первая половина координат играет роль «x», вторая — «y»
11    seq_len, dim = x.shape
12    x1, x2 = x[:, :dim // 2], x[:, dim // 2:]
13    rotated = torch.cat((-x2, x1), dim=-1) # поворот пары: (x, y) -> (-y, x)
14    return x * cos[:seq_len] + rotated * sin[:seq_len] # классическая формула поворота
```

Проверяем свойство на нашем примере

query_2 и keys_2 из раздела про три матрицы — на всех шести позициях

```
1 cos, sin = compute_rope_params(dim=2, context_length=6)
2 q_rep = query_2.repeat(6, 1)      # запрос второго токена — на всех позициях
3 k_rep = keys_2.repeat(6, 1)       # его же ключ — на всех шести позициях
4 q_rot, k_rot = apply_rope(q_rep, cos, sin), apply_rope(k_rep, cos, sin)
5 print(q_rot[1] @ k_rot[3], q_rot[2] @ k_rot[4]) # tensor(-0.6314) tensor(-0.6314)
6 print(q_rot[0] @ k_rot[5])         # tensor(0.3784) — расстояние 5
```

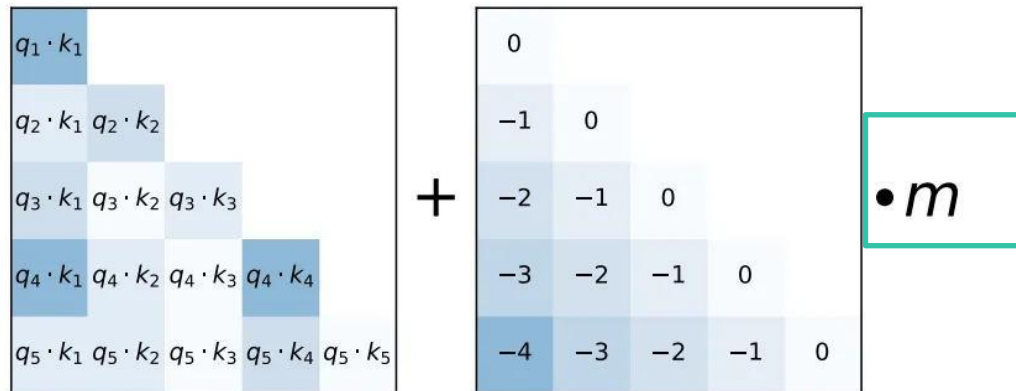
Штраф за расстояние вместо позиции

Матрица штрафов строится одной строкой

```
1 | T = 5
2 | dist = torch.arange(T)[: , None] - torch.arange(T)[None, :] # разность i - j
3 | penalty = -dist.clamp(min=0) # штраф растёт с расстоянием: 0, -1, -2
4 | print(penalty)
5 | # tensor([[ 0,  0,  0,  0,  0],
6 | #         [-1,  0,  0,  0,  0],
7 | #         [-2, -1,  0,  0,  0],
8 | #         [-3, -2, -1,  0,  0],
9 | #         [-4, -3, -2, -1,  0]]) # нули выше диагонали закроет causal mask
```

ALiBi (attention with linear biases): позиция не кодируется вовсе — штраф прибавляется к attention scores.

Штраф прибавляется к показателям



- Наклон m — множитель штрафа, свой у головы
- Позиции как вектора нет — любая длина входа

Относительность RoPE даёт применимость к любой длине, но не качество: на длинах много больше обученной он деградирует (NTK-scaling, YaRN — Peng et al., 2023, arXiv:2309.00071).

- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
- attention scores — числа до нормировки: скалярные произведения query и key
- causal mask — запрет смотреть вперёд: $-\infty$ над диагональю до softmax

Источник: Рис. 1.3.20 · Press et al., 2021, репозиторий авторов ALiBi, github.com/ofirpress/attention_with_linear_biases

Для самостоятельного чтения

В домашнем задании — класс каузального внимания и сверка с эталоном; RoPE и ALiBi писать не нужно

- Наклоны m у ALiBi: как подбирают штраф для каждой головы
- Границы относительных схем: где «важна только разность» перестаёт выполняться
- Перечень схем целиком: видов оказывается пять, полнота — через собственные значения

$$A(\tau_1 + \tau_2) = A(\tau_1)A(\tau_2)$$

$A(\tau)$ — матрица для расстояния τ (tau): два сдвига подряд дают тот же результат, что один на их сумму

- query / key / value — три линейные проекции эмбединга: что ищу, чем себя объявляю, что отдаю
attention scores — числа до нормировки: скалярные произведения query и key
causal mask — запрет смотреть вперёд: $-\infty$ над диагональю до softmax

Итоги занятия

- Три шага: scores, softmax, context vector
- Три матрицы дают query, key и value из одного эмбединга
- Деление на корень из d_k спасает softmax от насыщения
- Causal mask — до softmax, иначе строки не дают единицу
- Отладка: формы, суммы строк, масштаб, сверка с эталоном
- Позиция входит в query и key: RoPE или ALiBi

Куда читать дальше

→ ALiBi, Press et al., 2021: arxiv.org/abs/2108.12409

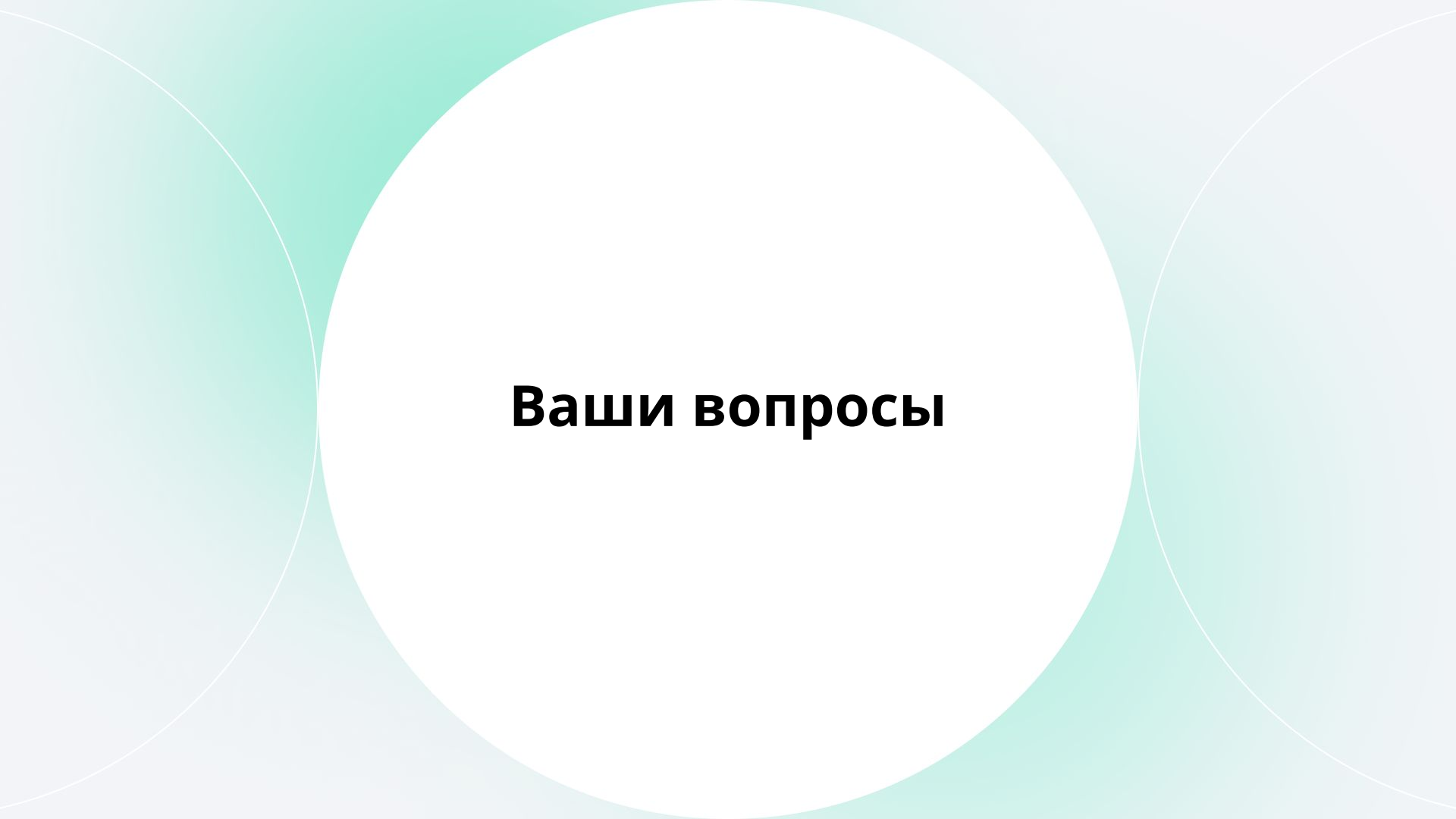
→ Реализация ALiBi у авторов:
github.com/ofirpress/attention_with_linear_biases

→ Positional encodings в 2025 году: habr.com/ru/articles/926368/

Перечень позиционных схем, Jane Street:
→ blog.janestreet.com/using-group-theory-to-explore-positional-encodings-attention/

➤ Kazemnejad et al., 2023. NoPE: arxiv.org/abs/2305.19466

➤ Peng et al., 2023. YaRN: arxiv.org/abs/2309.00071



Ваши вопросы

Входной конвейер в коде (scaffolded)

Self-attention: query, key, value и causal mask

Источники: С. Рашка. Создаём большую языковую модель с нуля; Vaswani et al., 2017. Attention Is All You Need

Александр Чурсин
Руководитель группы
разработки в ГК «Норильский
никель»

